

A Synchronised Home Automation Framework using IoT Ecosystem Primitives

Abstract

The growing need for convenience, energy saving, and increased security in home settings has triggered immense interest in smart home technology. The current project is to design and offer a cutting-edge Internet of Things (IoT)-based Smart Home Automation System designed to integrate and control a wide range of home appliances, ranging from traditional devices like lights and fans to advanced sensor-based lighting schemes. The central function of this system empowers the user with full remote control, made available through user-friendly web-based user Interface. This shift from more conventional, frequently localized forms of control presents a wide range of advantages, such as increased user convenience, the optimal use of energy by means of smart automation, and an increased perception of safety within the home.

Taking inspiration from current smart home systems and building on developments in IoT communication protocols and sensor technologies, our system provides a scalable, interoperable, and user-friendly solution for smart home automation. Intelligent automation based on real-time information and user preferences, coupled with successful integration of heterogeneous device types, is a major milestone towards the development of autonomously intelligent and adaptive living environments. Having established and now operating a working prototype, future efforts will be directed toward systematic performance testing and user feedback analysis to enhance the system's robustness and usability. Subsequent research will investigate the integration of more sophisticated features like predictive analytics for proactive device management, increased interoperability with more diverse smart building systems and third-party services, and optimization for energy efficiency based on actual usage patterns. The potential shown to be delivered by this work is in its capacity to enable users to have more control over their home space, lower energy usage, and improve quality of life through smart automations

Index

Chapter No.	Topic	Page No.
	List of Figures	1
	List of Abbreviations	2
01	PROJECT DESCRIPTION AND OUTLINE 1.1 Introduction 1.2 Motivation for the Work 1.3 Problem Statement 1.4 Objective of the Work 1.5 Organization of Thesis	5
02	LITERATURE SURVEY 2.1 Introduction 2.2 Core Area of the Project 2.3 Existing Algorithms and Systems 2.3.1 Early Wired Automation Systems 2.3.2 IoT-Enabled Automation Systems 2.3.3 Middleware for Interoperability 2.3.4 Energy Management Systems 2.4 Research Issues/Observations from Literature Survey 2.5 Summary	7
03	PROJECT PROCEDURE 3.1 Introduction 3.2 System Architecture 3.3 Flowchart	9

	3.4 Module 1: Device Connectivity and Control 3.2.1 Design 3.3.2 Implementation 3.5 Module 2: Data Processing and Decision Making 3.5.1 Design 3.5.2 Implementation	
04	WORK DONE 4.1 Introduction 4.2 Component Used 4.3 System Working 4.4 Code Flow and Implementation Summary	14
05	CONCLUSION AND FUTURE SCOPE	37
	References	39
	Appendix A	43

LIST OF FIGURES

Fig. No.	Name	Page No.
1.	ESP 32	10
2.	Bread Board	10
3.	Jumper Cables	10
4.	Raspberry Pi	10
5.	DHT11	10
6.	LED	10
7.	Resistor	10
8.	12V Fan	11
9.	Relay Module	11
10.	IoT Home Automation System Workflow	11
11.	LED OFF from Blynk Website	12
12.	LED ON from Blynk Website	12
13.	LED Control via Blynk App Workflow	13
14.	Raspberry Pi 4 Model B	17
15.	PIR Motion Sensor	18
16.	The Pyroelectric Sensor	18
17.	The Pyroelectric Sensor sensing	19
18.	The Fresnel Lens sensing and forming the graph	19
19.	The Fresnel Lens working	19
20.	DHT11 Sensor Pin out Senesor	20
21.	DHT11 Sensor PCB	20
22.	DHT11 Sensor Internal Structure	20
23.	NTC Thermistor with Characteristic Curve	21
24.	Relay Module	21
25.	Ultrasonic Sensor	22
26.	HC-SR04 Ultrasonic Sensor Pinout	22
27.	When obstetrical is there	23
28.	When No obstetrical is there	23
29.	TM1637 Pin Diagram	24
30.	TM1637 Display Circuit Backside	24
31.	AC Dimmer Control	25
32.	Circuit Diagram	25
33.	Graph of Different Cases	25
34.	Output voltage waveform of a phase-controlled AC dimmer	26
35.	Waveforms of an current	26
36.	The original AC wallpower waveform with the modified "chopped" AC waveform	26

37.	Oscilloscope display of waveforms	26
38.	Light Dependent Resistor (LDR)	27
39.	Light Dependent Resistor (LDR) Working	25
40.	Relay Module	25
41.	Relay Module Pin Diagram	25
42.	Relay Module Circuit diagram	26
43.	Motion Detection Logic	26
44.	Temperature and Humidity Monitoring	27
45.	Temperature and Humidity Monitoring Working	27
46.	Web LED Control	28
47.	AI-powered chat assistant using LangChain and Groq	32
48.	System Initialization & Core Functions	34
49.	Additional Sensor-Based Functionalities	34
50.	Smart Automation & Notification Features	34
51.	Sequence Diagram	35

LIST OF ABBREVIATIONS

1. **LED** - Light-emitting diode
2. **IoT** - Internet of Things
3. **Wi-Fi** - Wireless Fidelity
4. **ESP32** - Espressif Systems 32-bit microcontroller
5. **Raspberry Pi** - Single-board computer
6. **Blynk** - A platform to build mobile applications for the Internet of Things
7. **DHT11** - Digital Humidity and Temperature sensor
8. **GPIO** - General Purpose Input/Output
9. **Wi-Fi** - Wireless Fidelity
10. **HTTP** – Hyper Text Transfer Protocol
11. **API** - Application Programming Interface
12. **MQTT** - Message Queuing Telemetry
13. **JSON** - JavaScript Object Notation
14. **HTTP** – Hyper Text Transfer Protocol
15. **AJAX** - Asynchronous JavaScript and XML
16. **jQuery** - JavaScript library
17. **MQTT** - Message Queuing Telemetry Transport
18. **URL** - Uniform Resource Locator
19. **VNC** - Virtual Network Computing
20. **USB** - Universal Serial Bus
21. **PIR** – Passive Infrared Sensor
22. **LED** – Light Emitting Diode
23. **DC** – Direct Current
24. **LDR** – Light Dependent Resistor
25. **DHT11** – Digital Humidity and Temperature Sensor
26. **DFD** – Data Flow Diagram

LIST OF TABLES

Fig. No.	Name	Page No.
01	Specification of Ultrasonic Sensor	18
02	TM1637 Display Specification	24
03	Specification table of LDR Sensor	27

Chapter 1

PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

Modern life has become more connected and technology-driven. In a typical household, various devices like lights, air conditioners, televisions, and security cameras are used daily. Managing all these devices individually can be cumbersome. This is where a smart home automation system comes in.

A smart home system uses Internet of Things (IoT) technology to connect and control household devices. IoT allows these devices to "talk" to each other and to users through mobile apps, voice commands, or automation. For example, you can set the lights to turn on automatically when it gets dark or have your air conditioner adjust its temperature based on the weather.

This system not only adds convenience but also helps save energy and enhances home security^[1]. For instance, motion sensors can detect intruders and send alerts to your phone. Smart plugs can turn off devices when not in use, saving electricity.

Despite these benefits, current systems often face challenges, such as difficulty in integrating devices from different brands or being too expensive for the average user. Our project aims to create a system that is affordable, easy to set up, and compatible with a wide range of devices.

1.2 Motivation for the Work

With increasing demands for energy conservation, convenience, and safety, traditional home automation systems often fall short due to their cost, complexity, and lack of compatibility across devices. IoT-based systems address these challenges by offering affordable and scalable solutions that work with a wide variety of devices. This project aims to make home automation accessible for the average user while ensuring security and reliability ^[2].

1.3 Problem Statement

Current home automation systems face several limitations:

- Lack of compatibility between devices from different manufacturers.
- Security vulnerabilities, making user data susceptible to breaches.
- High setup costs, restricting adoption by middle-income households.
- Limited scalability, with difficulties integrating new devices.

This project seeks to overcome these issues by designing an IoT-based system that is secure, flexible, and easy to install.

1.4 Objective of the Work

The project aims to design a user-friendly smart home automation system using IoT technology, which ensures:

1. Seamless integration of devices using common protocols like MQTT and Wi-Fi.
2. Secure communication to protect user data.
3. Cost-effective hardware solutions such as Raspberry Pi or ESP32 for scalability.
4. Features such as remote monitoring and control via a mobile app or voice assistant.

1.5 Organization of Thesis

- **Chapter 1:** Background, motivation, and objectives of the project.
- **Chapter 2:** Literature review of existing systems and their limitations.
- **Chapter 3:** Design and implementation details, including the system architecture, modules, and their functioning.
- **Chapter 4:** Results, discussions, and individual contributions.
- **Chapter 5:** Conclusion and future scope.

Chapter 2

LITERATURE SURVEY

2.1 Introduction

Smart home automation systems have evolved significantly over the years, transitioning from simple wired systems to sophisticated IoT-enabled solutions. This chapter explores past advancements, existing systems, and the challenges they face, highlighting the gaps in current technology that this project aims to address.

2.2 Core Area of the Project

The project focuses on developing an IoT-based smart home system that integrates devices such as sensors, actuators, and controllers. It emphasizes interoperability, data security, and scalability. The core areas include:

- **Interoperability:** Ensuring compatibility between devices using protocols like MQTT and Wi-Fi.
- **Data Security:** Protecting sensitive user information from unauthorized access.
- **Scalability and Cost-Effectiveness:** Allowing the addition of new devices without significant costs or infrastructure changes.

2.3 Existing Algorithms and Systems

2.3.1 Early Wired Automation Systems

The initial home automation systems were primarily wired. These systems provided limited flexibility as they required significant physical infrastructure changes for installation and upgrades. Furthermore, they were expensive, making them inaccessible for most households. The reliance on proprietary technologies further limited their scalability and compatibility with newer devices ^{[1][2]}.

2.3.2 IoT-Enabled Automation Systems

The advent of IoT introduced wireless communication protocols such as Zigbee, Z-Wave, and Wi-Fi, which significantly improved the accessibility and functionality of home automation systems. These systems allowed real-time monitoring and control of household devices through mobile applications, offering unprecedented convenience and efficiency ^[3].

2.3.3 Middleware for Interoperability

Middleware platforms, such as Open Connectivity Foundation (OCF) and Matter, have been developed to address the lack of compatibility among devices from different manufacturers. These platforms provide a standardized framework for device communication, enabling seamless integration ^{[4][5]}.

2.3.4 Energy Management Systems

IoT-based energy management systems allow households to monitor and optimize their energy usage. For example, smart thermostats adjust heating and cooling systems based on occupancy and external weather conditions, reducing energy consumption ^[6].

2.4 Research Issues/Observations from Literature Survey

1. Interoperability Challenges:

Devices from different manufacturers often fail to communicate effectively due to the lack of standardized protocols. This creates "silos" within home automation systems, limiting their scalability ^[4].

2. Data Privacy and Security Risks:

Many IoT devices are prone to cyber-attacks due to inadequate security measures, posing risks to user data and overall system integrity ^[7].

3. High Initial Costs and Complexity:

While IoT has made home automation more accessible, the costs of high-end systems and the technical expertise required for setup remain significant barriers for widespread adoption ^{[3][8]}.

4. Reliability of Communication Protocols:

Wireless protocols like Zigbee and Wi-Fi can suffer from latency and interference, affecting system reliability in real-time applications ^[5].

2.5 Summary

The literature highlights the evolution of home automation from wired to IoT-enabled systems and identifies key challenges such as interoperability, security, and cost-effectiveness. These findings underscore the need for a comprehensive solution that integrates devices seamlessly while addressing these limitations.

CHAPTER 3

PROJECT PROCEDURE

3.1 Introduction

This chapter describes the design, implementation, and functioning of the proposed IoT-based smart home automation system. The system is divided into three key modules: device connectivity and control, data processing and decision-making, and cloud integration.

3.2 System Architecture

The system architecture is designed to ensure seamless communication between devices while prioritizing scalability and security. The architecture includes:

1. **User Interface:**
 - Mobile App: For remote control and monitoring.
 - Voice Assistants: For hands-free operation.
2. **Central Controller:**
 - Raspberry Pi or ESP32, which acts as the brain of the system, processing data and sending commands to devices.
3. **Sensors and Actuators:**
 - Sensors: Devices like motion, temperature, and light sensors collect environmental data.
 - Actuators: Devices like smart lights, locks, and thermostats perform actions based on commands.
4. **Cloud Server:**
 - Stores data, processes analytics, and provides insights.

Components Used:

1. **ESP32:** A versatile microcontroller with built-in Wi-Fi and Bluetooth capabilities, used for device control and communication.
2. **Breadboard:** For building and testing circuit connections.
3. **Jumper Cables:** For wiring components on the breadboard.
4. **12V Fan:** Represents a household appliance that can be controlled remotely.
5. **Raspberry Pi:** Functions as the central controller to process data and manage communication.
6. **DHT11 (Temperature Sensor):** Detects ambient temperature and humidity for climate automation.
7. **Light Emitting Diode (LED):** Used to demonstrate remote on/off functionality.
8. **Resistors:** To control current flow and protect components like LEDs.

9. **Relay Module:** Enables control of high-power appliances, such as the fan, through low-power circuits.

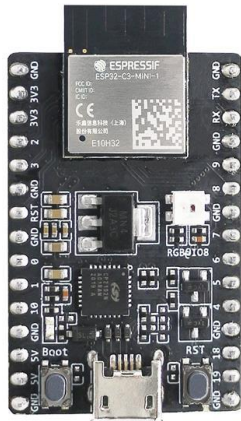


Fig No.1: - ESP32 Cables

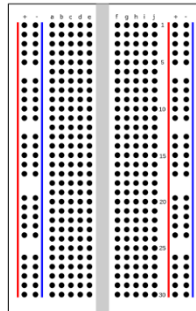


Fig No.2: - Bread Board



Fig No. 3: - Jumper

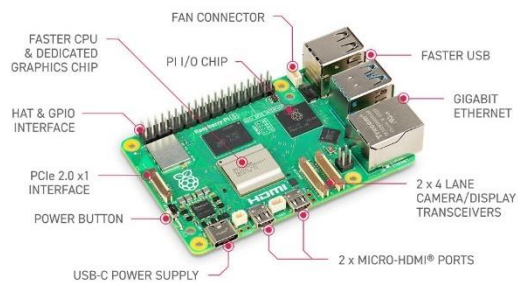


Fig No. 4: - Raspberry Pi

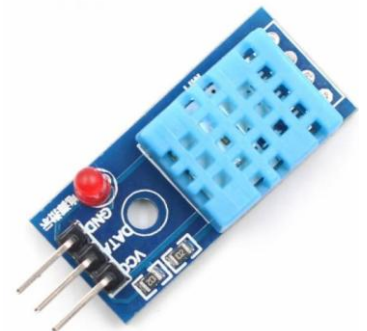


Fig No. 5: - DHT11



Fig No. 6: - LED

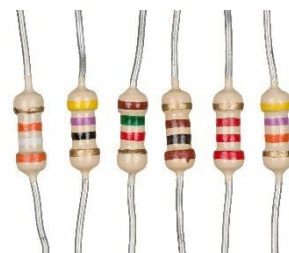


Fig No. 7: - Resistor



Fig No. 8: - 12V Fan



Fig No. 9: - Relay Module

3.3 Flowchart

The flowchart below provides a high-level overview of the system's workflow.

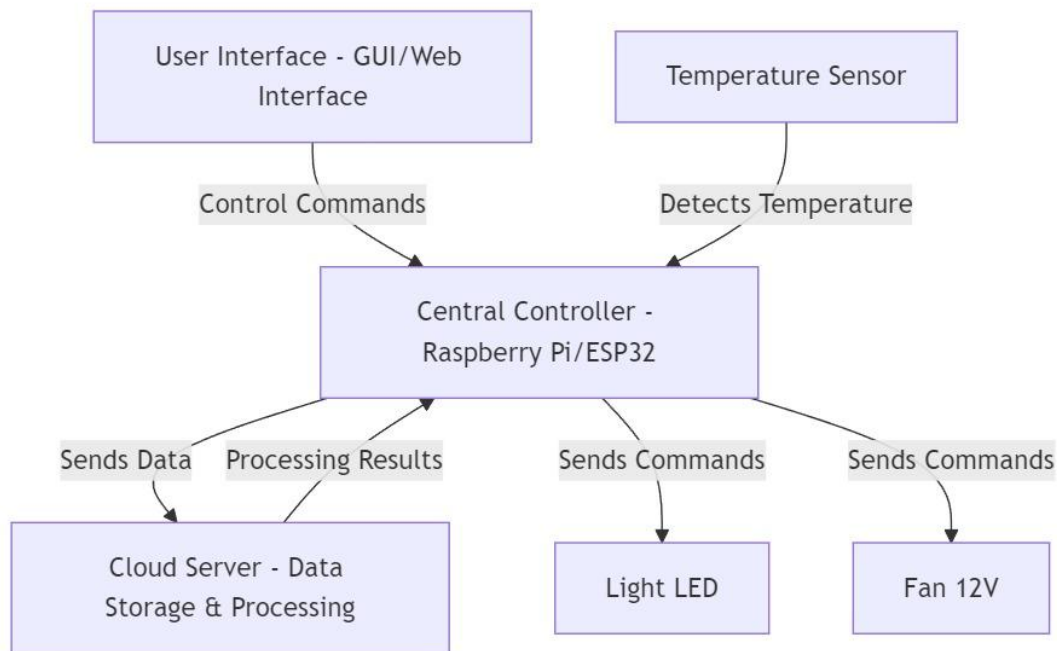


Fig. No. 10: - IoT Home Automation System Workflow

3.4 Module 1: Device Connectivity and Control

3.2.1 Design

This module ensures seamless communication between the ESP32 microcontroller, the LED, and the Blynk web interface. The ESP32 acts as the central controller, receiving commands from the Blynk website to control the LED's state.

The design is simple yet effective for understanding the basic functionality of IoT systems:

1. The Blynk website provides a user-friendly interface for sending ON/OFF commands.
2. The ESP32 receives these commands via Wi-Fi and processes them.
3. The LED serves as a visual indicator, switching ON or OFF based on the received commands.

3.3.2 Implementation

1. ESP32 Configuration:

- a. The ESP32 was programmed using the Arduino IDE and the Blynk library.
- b. Wi-Fi credentials were set up to connect the ESP32 to the internet and enable communication with the Blynk cloud server.

2. Blynk Mobile App Integration:

- a. A Blynk project was created, and a virtual button was added to the dashboard to control the LED.
- b. The button's state (ON/OFF) was linked to a virtual pin on the ESP32, which processed the input to toggle the LED.

3. LED Control Logic:

- a. When the button in the Blynk website is pressed, a signal is sent to the ESP32 through the Blynk cloud server.
- b. The ESP32 reads the signal and toggles the LED's state accordingly:
 - i. **ON:** The LED lights up.
 - ii. **OFF:** The LED is turned off.
- c. A confirmation signal is sent back to the Blynk app, allowing real-time status updates.

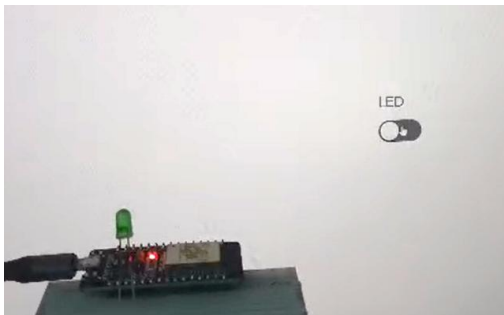


Fig. No. 11: - LED OFF from Blynk Website

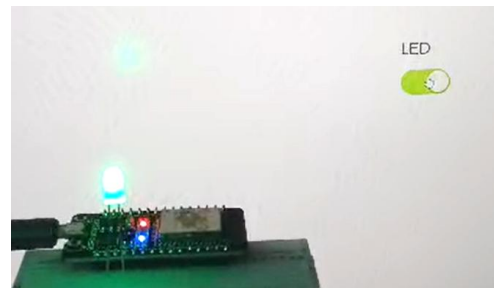


Fig. No. 12- LED ON from Blynk Website

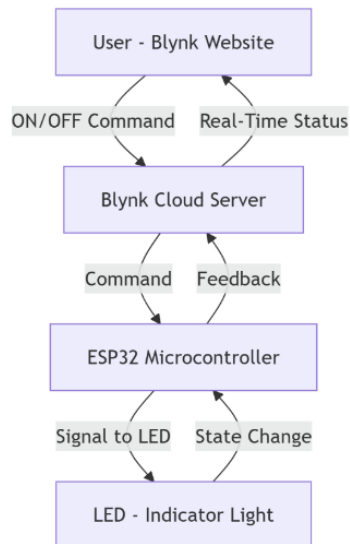


Fig. No. 13: - LED Control via Blynk App Workflow.

3.5 Module 2: Data Processing and Decision Making

3.5.1 Design

This module focuses on the processing of commands received from the Blynk app and ensuring the LED operates as per user input. The ESP32 monitors the virtual pin connected to the Blynk app and executes the appropriate action based on the received data.

3.5.2 Implementation

- **Logic for ON/OFF Commands:**
 - The ESP32 continuously checks for input on the designated virtual pin.
 - If the received command is "1" (ON), the ESP32 sends a HIGH signal to the GPIO pin connected to the LED.
 - If the received command is "0" (OFF), the ESP32 sends a LOW signal, turning the LED off.
- **Error Handling:**
 - To ensure reliability, the system validates the Wi-Fi connection and the command received from the Blynk server.
 - In case of disconnection, the ESP32 attempts to reconnect automatically, ensuring seamless operation.

CHAPTER 4

WORK DONE

4.1 Introduction

The successful implementation of the Home Automation System using Raspberry Pi requires the integration of various electronic components and sensors. This chapter provides a detailed explanation of the core components used in the project along with their working principles, technical specifications, and practical applications.

4.2 Components Used

4.2.1 Raspberry Pi

The Raspberry Pi is a small, powerful computer that acts as the central processing unit for our smart home automation system. It is capable of interfacing with various sensors and modules, making it ideal for IoT applications.

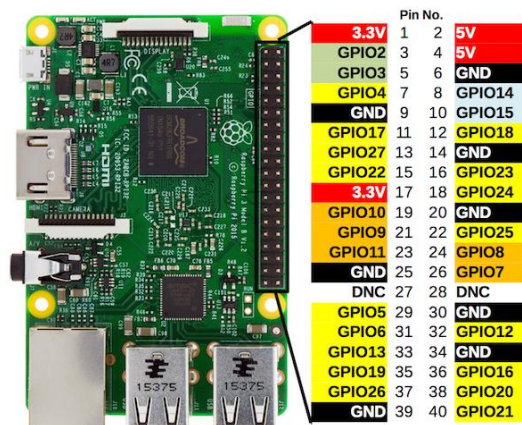


Figure No 14: Raspberry Pi 4 Model B

4.2.2 PIR Motion Sensor

The Passive Infrared (PIR) Motion Sensor is used to detect motion by sensing infrared radiation emitted by objects. In this project, the PIR sensor is responsible for detecting human presence and triggering the LED.

A basic infrared receiver can detect the presence of infrared radiation, but to detect movement, we need something that can notice changes in infrared levels. This is exactly what PIR (Passive Infrared) sensors do! They're specially designed to detect changes in infrared radiation caused when warm objects, like people, move within their detection range. ²³

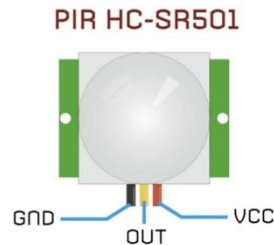


Figure No. 15: PIR Motion Sensor

VCC is the power supply pin for the sensor. You can connect this pin to a power source that provides anywhere from 5 volts to 12 volts. Most people use 5 volts because that's what's commonly available from an Arduino or similar microcontroller.

Output is the 3.3V TTL logic output pin. When the sensor detects movement, this pin goes HIGH (3.3 volts). When there's no movement detected, the pin stays LOW (0 volts).

GND is the ground pin.

A PIR sensor has two main parts:

1. Pyroelectric sensor – This is the heart of the device, appearing as a round metal piece with a rectangular crystal in the center.
2. Fresnel lens – A special lens that collects and focuses infrared signals onto the pyroelectric sensor.

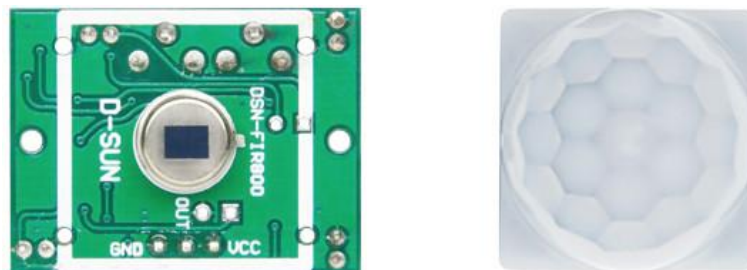


Figure No 16: The Pyroelectric Sensor

The pyroelectric sensor has a window with two rectangular slots made from a material (usually silicon with a special coating) that allows infrared radiation to pass through. Behind this window are two separate infrared-detecting electrodes – one produces a positive output, while the other produces a negative output.

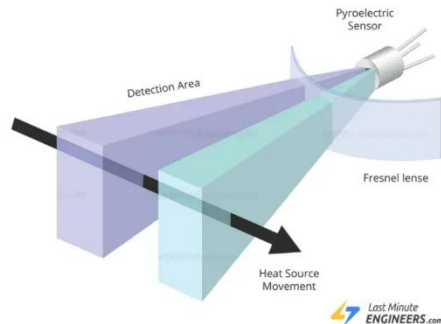


Figure No.17: The Pyroelectric Sensor sensing

When nothing is moving near the sensor, both parts detect the same amount of infrared radiation, so they cancel each other out and no signal is produced.

However, when a warm object like a person moves within the sensor's detection area, one part detects a change in infrared radiation before the other. This creates a difference in signals between the two parts, which the sensor recognizes as movement.

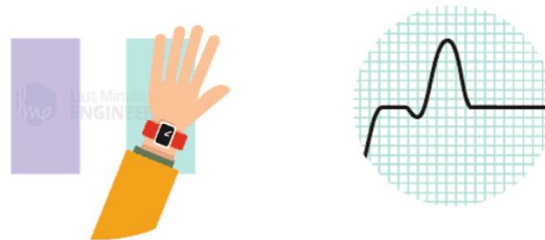


Figure No. 18: The Fresnel Lens sensing and forming the graph

The Fresnel lens might not look like it's doing much, but it actually greatly increases the range and field of view of the PIR sensor.

A Fresnel lens is made of plastic with a series of concentric circular grooves carved into it. Each groove acts as a refracting surface that focuses parallel light rays onto a focal point, similar to how a regular optical lens works, but in a much flatter design.¹³

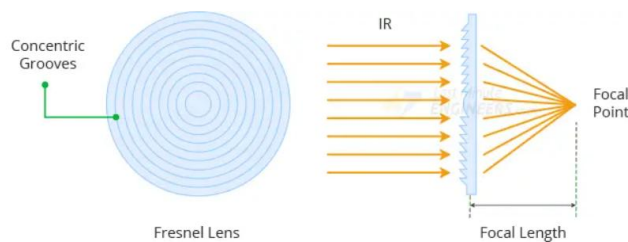


Figure No 19: The Fresnel Lens working

To increase the detection range and field of view, the lens is divided into multiple sections, with each section acting as its own individual Fresnel lens.

4.2.3 LED (Light Emitting Diode)

The LED is used as a visual indicator in our system. It turns ON when motion is detected and turns OFF when no motion is detected.

4.2.4 DHT11 Temperature and Humidity Sensor

The DHT11 sensor is responsible for measuring the temperature and humidity of the surrounding environment. It communicates with the Raspberry Pi through GPIO pins to provide real-time data. DHT11 can measure temperature from 0°C to 50°C with a $\pm 2.0^\circ\text{C}$ accuracy, and humidity from 20 to 80% with a 5% accuracy.³⁸



Figure No. 20: DHT11 Sensor Pin out Sensor

The DHT11 module is relatively simple to connect. There are only three pins:

+ (VCC) pin provides power to the sensor. Despite the fact that the supply voltage of the module ranges from 3.3V to 5.5V, a 5V supply is recommended. With a 5V power supply, the sensor can be placed up to 20 meters away. With 3.3V supply voltage, the sensor can be placed just 1 meter away; otherwise, the line voltage drop will cause measurement errors.

Out pin is used for communication between the sensor and the microcontroller.

– (GND) is the ground pin.

Note that the DHT11 has a sampling rate of 1Hz, which means it can provide new data once every second.

The module includes all of the necessary supporting circuitry, so it can be used straight out of the box.

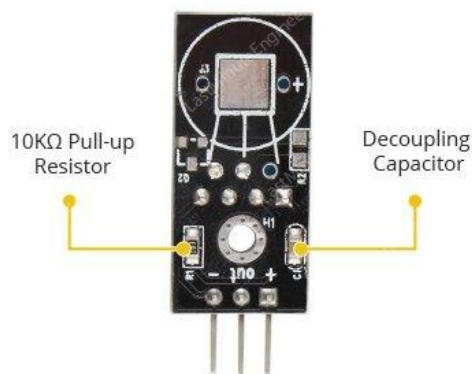


Figure No.21: DHT11 Sensor PCB

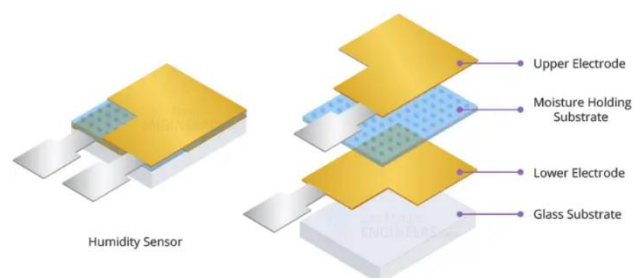


Figure No 22: DHT11 Sensor Internal Structure

DHT11 sensors typically require an external 10K pull-up resistor on the output pin for proper communication between the sensor and the Arduino. However, because the module already includes a pull-up resistor, you do not need to add one.

The humidity sensing component has two electrodes with a moisture-holding substrate (usually a salt or conductive plastic polymer) in between.

As the humidity rises, the substrate absorbs water vapor, resulting in the release of ions and a decrease in the resistance between the two electrodes.

This change in resistance is proportional to the humidity, which can be measured to estimate relative humidity.

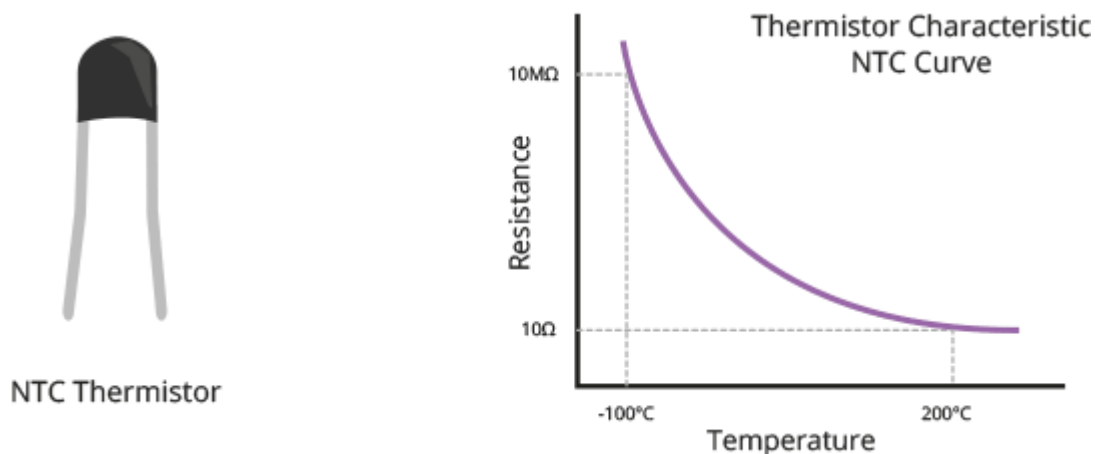


Figure No. 23: NTC Thermistor with Characteristic Curve

4.2.5 Relay Module

A relay module is used to control high-voltage devices using low-voltage signals from the Raspberry Pi. In our case, the relay can be used to control household appliances.



Figure No. 24: Relay Module

4.2.6 Breadboard and Jumper Wires

The breadboard and jumper wires are utilized as fundamental tools for prototyping and experimentation during the development of the home automation system. They allow for the creation of temporary electrical connections between different electronic

components without the need for soldering, thus facilitating rapid iteration and modification of the circuit designs.^[1]

4.2.7 Ultrasonic Sensor and Buzzer

The ultrasonic sensor operates on the principle of emitting high-frequency sound waves (ultrasound) and then measuring the time it takes for these waves to reflect off an object and return to the sensor.¹ This time measurement is directly proportional to the distance to the object, a principle similar to that used in SONAR and RADAR systems.¹ The HC-SR04, a common type of ultrasonic sensor, typically transmits sound waves at a frequency of 40 kHz.¹ To initiate a distance measurement, a short 10 μ s HIGH pulse is applied to the Trig pin of the sensor.¹ Upon receiving this trigger, the sensor emits an ultrasonic burst and simultaneously raises its Echo pin to HIGH.² The Echo pin remains HIGH until the reflected sound wave is detected by the sensor's receiver, at which point it returns to LOW. The duration for which the Echo pin is HIGH represents the time taken for the sound wave to travel to the object and back, allowing the distance to be calculated based on the speed of sound in air.¹ A longer duration indicates a greater distance, while a shorter duration signifies proximity.⁷



Figure No. 25 : Ultrasonic Sensor

Specification	Value
Operating Voltage	5V DC (some models support 3.3V)
Working Current	~15mA
Ranging Distance	2cm to 400cm (up to 4m in some cases)
Accuracy	~3mm
Measuring Angle	<15° or 30°

Table No 01: Specification of Ultrasonic Sensor

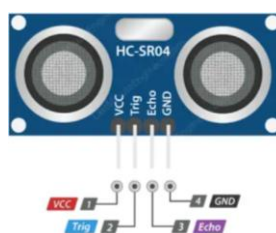


Figure No 26: HC-SR04 Ultrasonic Sensor Pinout

Now, let's take a closer look at the pinout of the HC-SR04 Ultrasonic Sensor and what each pin does:

1. VCC is the power pin for the sensor. You can connect this to the 5V output from your Arduino.
2. Trig (Trigger) is a signal input pin. When you want the sensor to measure distance, you pull this pin HIGH for 10 microseconds. When this pin receives that signal, the sensor responds by sending out 8 pulses of ultrasonic sound waves at 40 kHz.
3. Echo is a signal output pin. After the sensor sends out sound waves, this pin goes HIGH. The sensor then waits for those waves to bounce off an object and return. During this waiting period, the Echo pin stays HIGH. As soon as the sound waves bounce back and reach the sensor, the Echo pin goes LOW. The amount of time the Echo pin stays HIGH directly relates to how far away the object is. By measuring this time, your Arduino can calculate the exact distance to the object.
4. GND is the ground connection. Make sure to connect this to the ground pin of your Arduino to complete the electrical circuit.

Working

The HC-SR04 ultrasonic sensor measures distance using sound waves, like a bat. When the Trig pin is set HIGH for 10 μ s, the sensor emits an 8-cycle 40 kHz sound burst. The Echo pin then goes HIGH and stays HIGH until the sound reflects off an object and returns. The duration of this HIGH signal indicates the distance, ranging from 150 μ s to 25 ms.

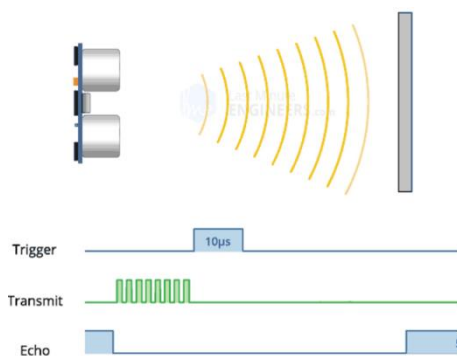


Figure 27 : When obstetrical is there

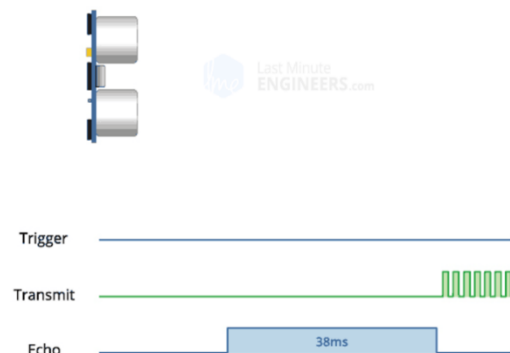


Figure 28: When No obstetrical is there

If the sound waves don't bounce back, the sensor will stop waiting after 38 ms. This timeout tells you there's nothing within the sensor's detection range of about 13 feet.

$$\text{Distance} = (0.034 \text{ cm}/\mu\text{s} \times 500 \mu\text{s}) \div 2$$

4.2.8 TM1637 Display

TM1637 display is a compact 4-digit 7-segment LED display module incorporated to provide a visual output for sensor readings or system status.³⁶ This module utilizes the TM1637 driver IC, which functions as an LED controller with a built-in keyboard scan interface.³⁷ Communication with the Raspberry Pi is achieved through a two-wire serial interface consisting of a clock pin (CLK) and a data input/output pin (DIO).³⁶ While the interface uses two wires, similar to I2C, it is important to note that the protocol is not entirely compatible with standard I2C communication.³⁷ The microcontroller sends serial data to the TM1637 module to display the desired characters. Key features of the TM1637 include the ability to adjust the display's brightness across eight different levels and the capability to show alphanumeric characters.³⁶



Figure 29: TM1637 Pin Diagram

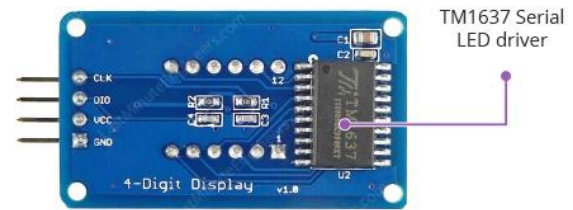


Figure 30 : TM1637 Display Circuit Backside

The TM1637 handles all the work of refreshing the display after it has been updated by the microcontroller, which is a nice bonus. This frees up the microcontroller to do other important things.^[19]

Another plus is that the TM1637 has many useful features, such as the ability to adjust the display's brightness and control each segment independently.

The TM1637 module operates on a 3.3 to 5 volt supply voltage and communicates via a two-wire bus, requiring only two data pins, VCC and ground. The TM1637 has its own proprietary data transfer protocol, but there are Arduino libraries available that hide the complexities and make communication with the display easier.

Operating voltage	3.3 – 5 VDC
Maximum Current Draw	80mA
Reverse polarity protection	Yes
Display Size	42 mm x 24 mm (1.65" x .95")
Display Height	12 mm (0.47") (typical)
Character Height	9.14 mm (0.36")

Table No 02: TM1637 Display Specification

4.2.9 AC Dimmer Control

The AC dimmer control module has been incorporated into the system to enable the adjustment of the brightness of lights powered by alternating current. This control is achieved through the sophisticated techniques of zero-cross detection and precise timing of the firing of a TRIAC (Triode for Alternating Current).⁵⁴ Zero-cross detection is crucial as it identifies the precise moment when the AC voltage waveform crosses zero, providing a reliable synchronization point for controlling the TRIAC.⁵⁴ The TRIAC functions as a solid-state switch that can conduct current in both directions.⁶⁴ By delaying the activation of the TRIAC (i.e., the firing angle) after each zero-crossing, the amount of power delivered to the light can be precisely regulated, resulting in the desired dimming effect.⁵⁴ It is important to note that for effective dimming of LED lights using a TRIAC dimmer, dimmable LED bulbs specifically designed for this purpose must be used.⁴⁰

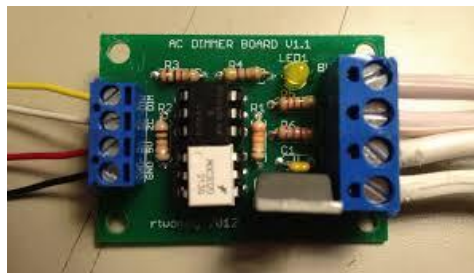


Figure No 31 : AC Dimmer Control

A Triac is a high-speed solid-state device that can switch and control AC power in both directions of a sinusoidal waveform

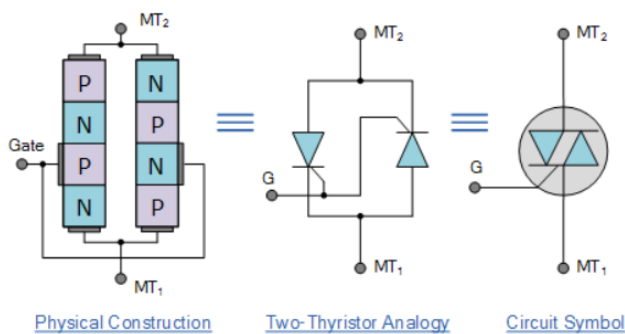


Fig No. 32: Circuit Diagram

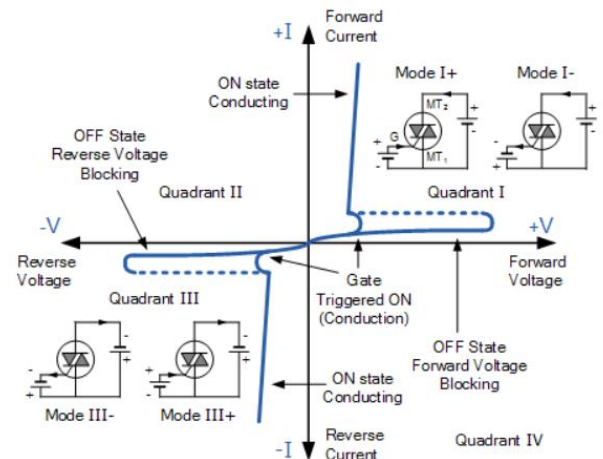


Fig No. 33: Graph of Different Cases

A Triac behaves just like two conventional thyristors connected together in inverse parallel (back-to-back) with respect to each other and because of this arrangement the two thyristors share a common Gate terminal all within a single three-terminal package.

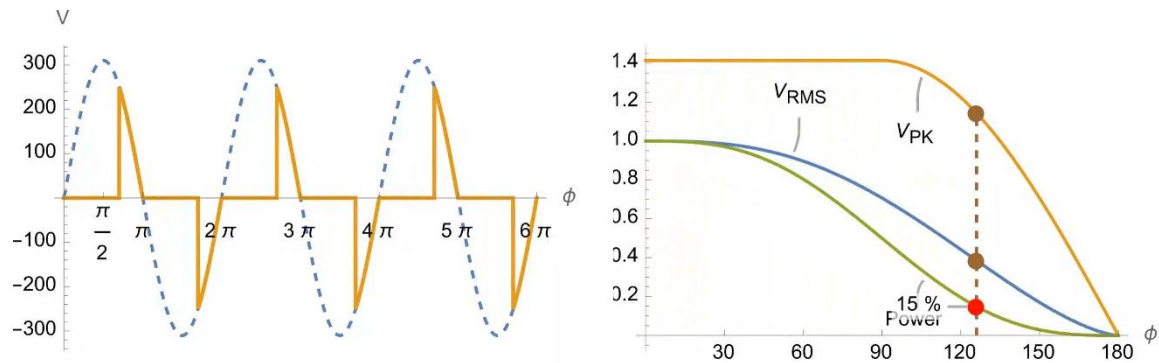


Fig No. 34: Output voltage waveform of a phase-controlled AC dimmer

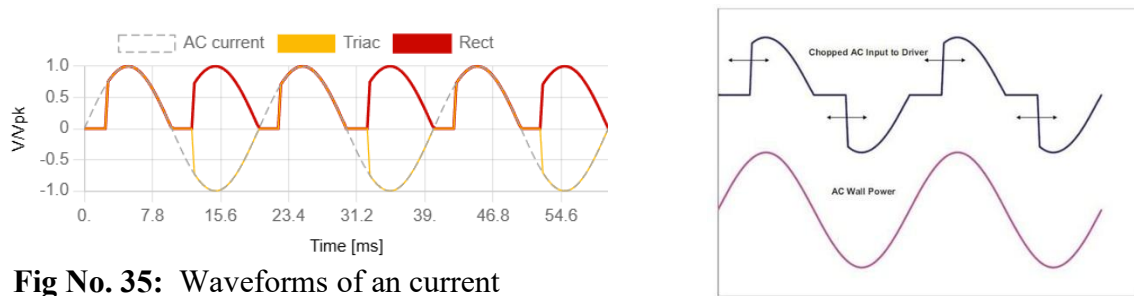


Fig No. 35: Waveforms of an current

Fig No.36:The original AC wallpower waveform with the modified "chopped" AC waveform

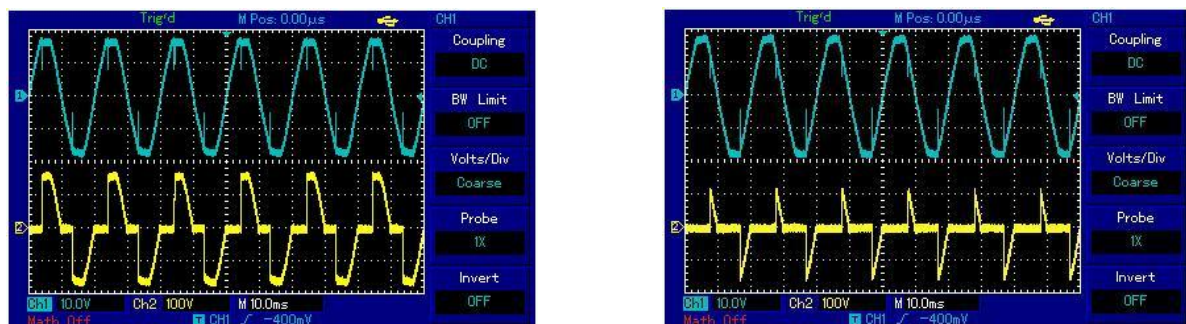


Fig No. 37: Oscilloscope display of waveforms

4.2.10 Weather Monitoring

The enhanced home automation system now incorporates the ability to fetch and display comprehensive weather information by utilizing the **OpenWeatherMap** API. This feature provides users with real-time weather data for a specified city, including current temperature, humidity levels, and a 5-day weather forecast, all presented through an intuitive graphical user interface developed using the Tkinter library.⁴⁰

4.2.11 Email Alert System

To further enhance the system's capabilities, an email alert system has been implemented. This system is designed to send automated email notifications to users based on specific predefined triggers or events occurring within the home automation system. This allows for effective remote monitoring and ensures that users are promptly alerted to important events, even when they are not physically present at their home.²⁶

SMTP Protocol: The backbone of this email alert system is the Simple Mail Transfer Protocol (SMTP), which is the standard communication protocol for electronic mail transmission on the internet.²⁶ SMTP is used by mail servers to send, receive, and relay outgoing email messages between senders and recipients.⁹

4.2.12 Light Dependent Resistor (LDR)

A Light Dependent Resistor (LDR), also known as a photoresistor, has been integrated into the system to detect variations in the ambient light levels.³⁵ The LDR operates based on the principle of photoconductivity, meaning that its electrical resistance changes in response to the amount of light falling on its surface.²⁹ Specifically, the resistance of the LDR decreases as the intensity of the incident light increases, and conversely, its resistance increases when the light level diminishes or in complete darkness.³⁷ LDRs are typically constructed using semiconductor materials such as cadmium sulfide (CdS).³⁵

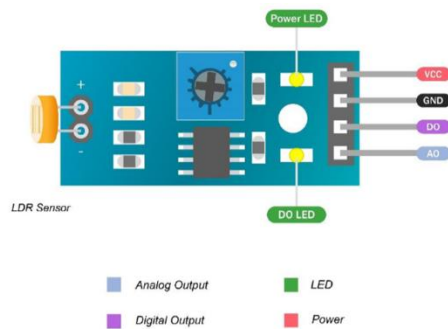


Figure No. 38: Light Dependent Resistor (LDR)

Max power dissipation	200mW
Max voltage @ 0 lux	200V
Peak wavelength	600nm
Min. resistance @ 10lux	1.8k Ω
Max. resistance @ 10lux	4.5k Ω
Typ. resistance @ 100lux	0.7k Ω
Dark resistance after 1 sec	0.03M Ω
Dark resistance after 5 sec	0.25M Ω

Table No. 03: Specification table of LDR Sensor

4.2.12 Relay

Relay is one kind of electro-mechanical component that functions as a switch. The relay coil is energized by DC so that contact switches can be opened or closed. A single channel 5V relay module generally includes a coil, and two contacts like normally open (NO) and normally closed (NC). This article discusses an overview of the 5V relay module & its working but before going to discuss what is relay module is, first we have to know what is relay and its pin configuration.

The pin configuration of the 5V relay module is shown below. This module includes 6-pins where each pin and its functionality are discussed below.³⁹

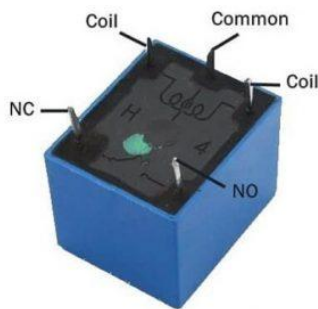


Figure No. 40:Relay Module



Figure No. 41: Relay Module Pin Diagram

Normally Open (NO): This pin is normally open unless we provide a signal to the relay modules signal pin. So, the common contact pin smashes its link through the NC pin to make a connection through the NO pin

Common Contact: This pin is used to connect through the load that we desire to switch by using the module.

Normally Closed (NC): This NC pin is connected through the COM pin to form a closed circuit. However, this NC connection will break once the relay is switched through providing an active high/low signal toward the signal pin from a microcontroller.

Signal Pin: The signal pin is mainly used for controlling the relay. This pin works in two cases like active low otherwise active high. So, in active low case, the relay activates once we provide an active low signal toward the signal pin, whereas, in an active high case, the relay will trigger once we provide a high signal toward the signal pin

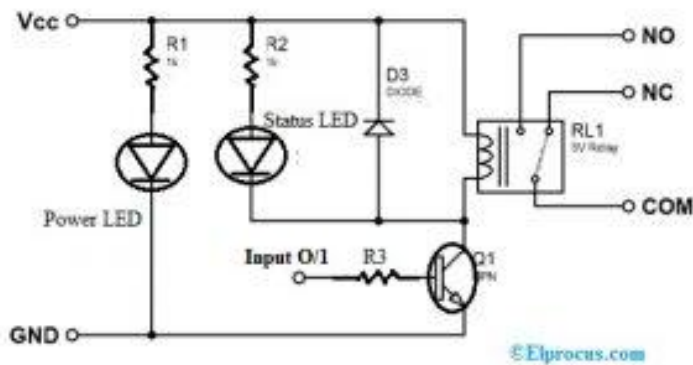


Figure No. 42: Relay Module Circuit diagram

4.3. System Working

The working of the system can be divided into the following sections:

4.3.1 Motion Detection

- The PIR motion sensor continuously monitors the surrounding environment for any movement. Upon detecting motion, it immediately sends a HIGH signal to the Raspberry Pi. This signal is then processed by the system to trigger a predefined action, in this case, turning the LED indicator ON. This functionality allows the system to react to the presence of occupants, which can be used for security purposes, automated lighting, or other presence-aware features. ^[29]

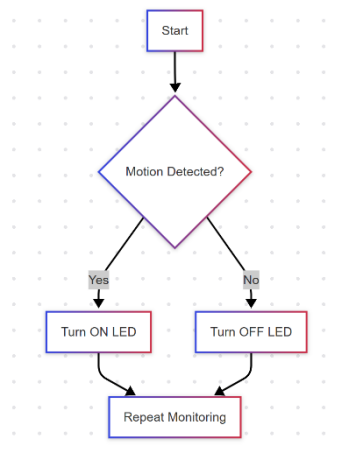


Fig No 43: Motion Detection Logic

4.3.2 Temperature and Humidity Monitoring

The DHT11 sensor plays a crucial role in gathering real-time data about the temperature and humidity of the environment. This sensor continuously collects these environmental readings and transmits them to the Raspberry Pi. The Raspberry Pi then processes this data and displays it on the web interface, providing users with a convenient way to monitor the conditions of their home remotely. This data is then displayed on the web interface.

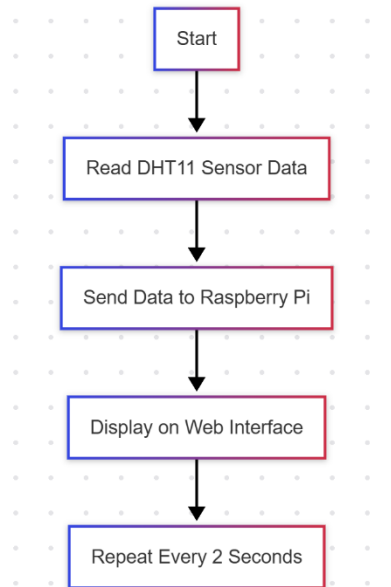


Fig No. 44: Temperature and Humidity Monitoring

4.3.3 LED Control from Web Interface

In addition to being triggered by motion detection, the LED indicator in the system can also be controlled directly by the user through the web interface. This web interface, built using the Flask framework, allows users to remotely turn the LED ON or OFF, providing a basic form of manual control over one of the system's output devices. This allows the user to turn the LED ON/OFF remotely.

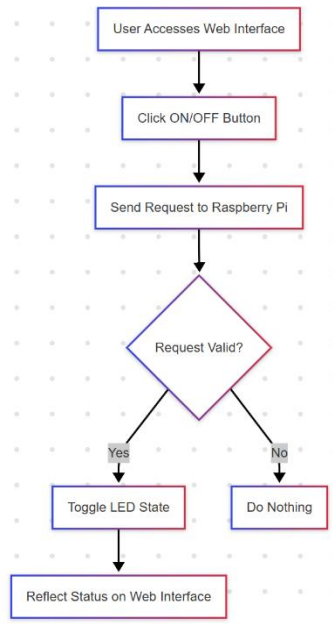


Fig No.46: Web LED Control

4.4 Additional System Functionalities

The enhanced home automation system now boasts several new functionalities that expand its capabilities significantly:

4.4.1 Distance Measurement and Alert:

The integration of an ultrasonic sensor enables the system to measure the distance to nearby objects. When the measured distance falls below a predefined threshold, the system triggers the connected buzzer to emit an audible alert. This feature can be utilized for applications such as obstacle detection for a mobile robot component (if integrated later) or as a proximity warning system.

4.4.2 Temperature Display:

The TM1637 display module is used to provide a direct visual readout of the current temperature. The temperature data, obtained from the DHT11 sensor, is processed by the Raspberry Pi and then displayed on the 4-digit 7-segment display, allowing users to quickly check the temperature without needing to access the web interface.

4.4.3 AC Dimmer Control:

Users of the system now have the ability to control the brightness of AC-powered lights that are connected through the AC dimmer control module. This functionality allows for adjusting the lighting levels to suit different needs or preferences, and it can also contribute to energy savings by dimming lights when full brightness is not required.

4.4.4 Weather Monitoring:

By leveraging the OpenWeatherMap API, the system can fetch and display current weather conditions, along with a 5-day forecast, for a specified city. This information is presented to the user through a graphical user interface, providing awareness of the external weather environment.

4.4.5 Email Alerts:

The system is now equipped to send email notifications for predefined events that occur within the home automation system. This feature allows for remote monitoring and ensures that users are alerted to important events, such as the detection of motion when the system is in a security mode, or perhaps unusual temperature or humidity readings.

4.4.6 Light-Based Automation:

The inclusion of a Light Dependent Resistor (LDR) allows the system to detect changes in ambient light levels. This capability can be used to automate actions based on the amount of natural light available, such as turning on a light when the room becomes dark or turning it off when sufficient daylight is present.

4.4.7 Face Recognition-Based LED Control:

This system leverages facial recognition technology to enable identity-based automation. By utilizing a camera feed processed through the `face_recognition` library, the system continuously scans for known individuals. When a specific individual (e.g., "Akshit") is detected in the frame, an LED connected via the GPIO interface is automatically activated. This provides a personalized response mechanism, such as granting access or signaling presence, based on real-time face detection. The implementation ensures efficient performance through downscaled image processing and includes real-time feedback via an on-screen FPS counter.

4.5 Code Flow and Implementation

The code is divided into three major sections:

1. **GPIO Setup and Initialization:** This section configures the Raspberry Pi GPIO pins.
2. **Sensor Data Reading:** This section reads input from the PIR and DHT11 sensors.
3. **Web Interface Control:** This section manages the web-based control using Flask.

4.5.1 GPIO Setup and Initialization

The initial section of the system's Python code is dedicated to configuring the General-Purpose Input/Output (GPIO) pins of the Raspberry Pi. This involves specifying the function of each pin that is connected to the various hardware components, such as setting pins as inputs for receiving data from sensors or as outputs for controlling LEDs, relays, the buzzer, and the AC dimmer. Proper setup and initialization of these GPIO pins are essential for ensuring that the Raspberry Pi can correctly communicate with and control all the connected peripherals. The following Python code is responsible for setting up the GPIO pins of the Raspberry Pi.

```
1 import RPi.GPIO as GPIO
2 GPIO.setmode(GPIO.BCM)
3 GPIO.setup(18, GPIO.OUT)
4 GPIO.setup(29, GPIO.IN)
5
```

Code Snippet: GPIO Setup

4.5.2 Sensor Data Reading

A significant portion of the code is responsible for reading the data generated by the different sensors integrated into the system. This includes the PIR motion sensor, the DHT11 temperature and humidity sensor, the ultrasonic sensor, and the LDR. The original chapter likely details the logic for reading the PIR sensor's digital output and the DHT11 sensor's temperature and humidity data. The code for reading data from the ultrasonic sensor (triggering it, measuring the echo pulse width, and calculating the distance) and the LDR (reading its analog value) is specifically provided in Appendix B, within Code Snippets B.1 and B.6, respectively. This section of the code ensures that the system can acquire the necessary information from its environment to perform its automated tasks and monitoring functions.

The sensor data reading happens in a loop where the Raspberry Pi continuously monitors for motion and reads temperature data.

```
1 while True:
2     if GPIO.input(29):
3         GPIO.output(18, GPIO.HIGH)
4     else:
5         GPIO.output(18, GPIO.LOW)
```

Code Snippet: PIR Sensor Reading

4.4.3 Web Interface Control

The web-based control of the LED, which allows users to remotely turn it ON or OFF, is managed through a web interface developed using the Flask microframework in Python. The core logic for handling user requests from the web interface and translating them into actions on the Raspberry Pi (such as changing the state of a GPIO pin connected to the LED) remains as described in the original chapter. This indicates that the fundamental mechanism for web-based interaction with the system has been established and is being leveraged for controlling at least one of the output components.

The web interface is created using Flask. It allows remote access to control the LED and view sensor data.

```
1 from flask import Flask, render_template
2 app = Flask(__name__)
3
4 @app.route('/')
5 def index():
6     return render_template('index.html')
7
8 @app.route('/led/on')
9 def led_on():
10     GPIO.output(18, GPIO.HIGH)
11     return 'LED ON'
12
13 @app.route('/led/off')
14 def led_off():
15     GPIO.output(18, GPIO.LOW)
16     return 'LED OFF'
17
18 if __name__ == '__main__':
19     app.run()
20
```

Code Snippet: Flask Web Interface

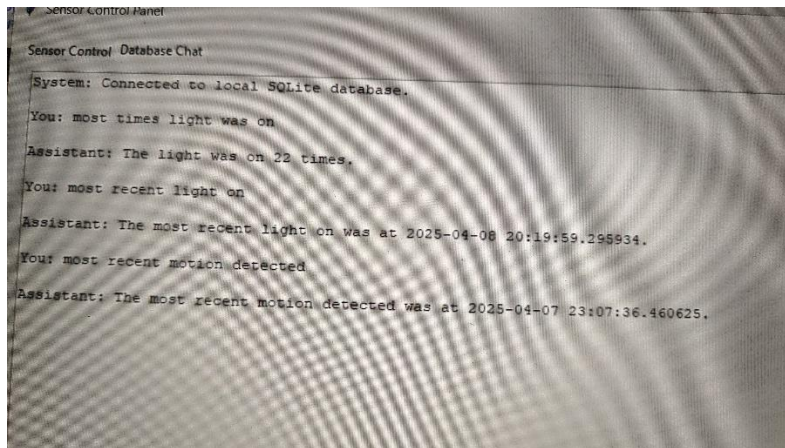


Fig. No.47: AI-powered chat assistant using LangChain and Groq

4.4.4 Additional Functionality Implementation

The implementation details for the new functionalities that have been added to the system are as follows:

4.4.4.1 Ultrasonic Sensor and Buzzer Control (Appendix A):

The code snippet provided in Appendix B, outlines the sequence of operations required to utilize the ultrasonic sensor for distance measurement. This includes sending a trigger pulse to the sensor, reading the duration of the echo pulse received back, using this duration to calculate the distance to an object, and then controlling the buzzer (likely by setting a GPIO pin connected to it HIGH or LOW) based on whether the measured distance is below a predefined threshold.

4.4.4.2 TM1637 Display Control (Appendix A):

Appendix B, Code, contains the Python code necessary for initializing the TM1637 display module and subsequently displaying the current temperature reading obtained from the DHT11 sensor on its 4-digit 7-segment interface. This would involve using a library specific to the TM1637 to send the appropriate commands and data to the display module via the CLK and DIO pins.

4.4.4.3 AC Dimmer Control (Appendix A):

The implementation of the AC dimmer control, as detailed in Appendix B includes the crucial aspects of zero-cross detection and the logic for firing the TRIAC at the appropriate time. This code would likely involve setting up an interrupt to detect the zero-crossing of the AC waveform and then using precise timing (potentially with software delays or hardware timers) to send a trigger signal to the TRIAC's gate, thereby controlling the brightness of the connected AC lights.

4.4.4.4 Weather Monitoring (Appendix A):

The code for fetching weather data from the OpenWeatherMap API and displaying it using the Tkinter graphical user interface is provided in Appendix B. This code would involve making HTTP requests to the OpenWeatherMap API endpoint, likely using a library like requests in Python, parsing the JSON

or XML response to extract the relevant weather information (temperature, humidity, 5-day forecast), and then using Tkinter widgets to present this data in a user-friendly window.

4.4.4.5 Email Alert System (Appendix A):

The function responsible for sending email alerts is detailed in Appendix B. This function would likely utilize the `smtplib` library in Python to connect to an SMTP server (such as Gmail's SMTP server), authenticate with the necessary credentials, construct the email message with the subject and body, and then send the email to the specified recipient(s) when a particular trigger event occurs within the home automation system.

4.4.4.6 LDR Sensor Control (Appendix A):

Appendix B contains the code for reading the light level detected by the LDR sensor. This likely involves reading an analog input pin on the Raspberry Pi to which the LDR is connected (often through a voltage divider circuit). The code then uses this light level reading to control an LED, for example, by turning it on when the light level falls below a certain threshold and off when it is above.

4.4.7.1 Face Recognition and LED Control (Appendix A):

The code snippet provided in Appendix C., demonstrates the integration of a face recognition system with GPIO-based hardware control. Utilizing the `face_recognition` library and a `PiCamera2` module, the system captures video frames in real time, processes them to detect and identify known faces based on preloaded encodings, and then controls an LED accordingly. Specifically, if the recognized face matches a predefined individual (e.g., "Akshit"), the GPIO pin connected to the LED is set HIGH to turn it on; otherwise, it remains LOW. This enables personalized hardware responses based on visual identification.

4.4.5 Data Flow Diagram (DFD)

The following Data Flow Diagram shows how data flows in our system.

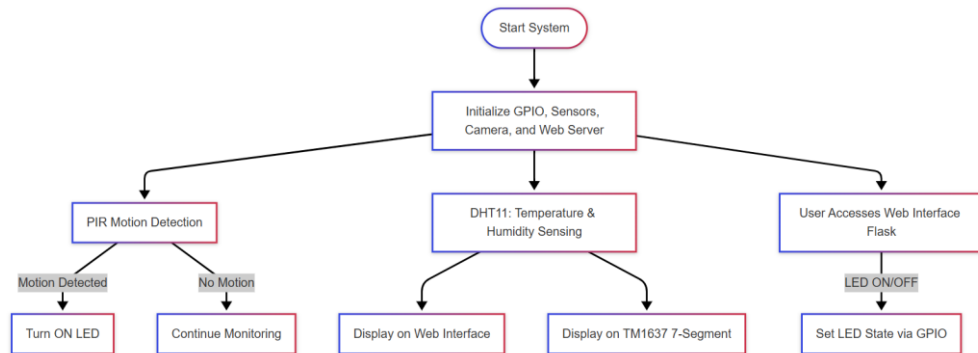


Fig No.48: System Initialization & Core Functions

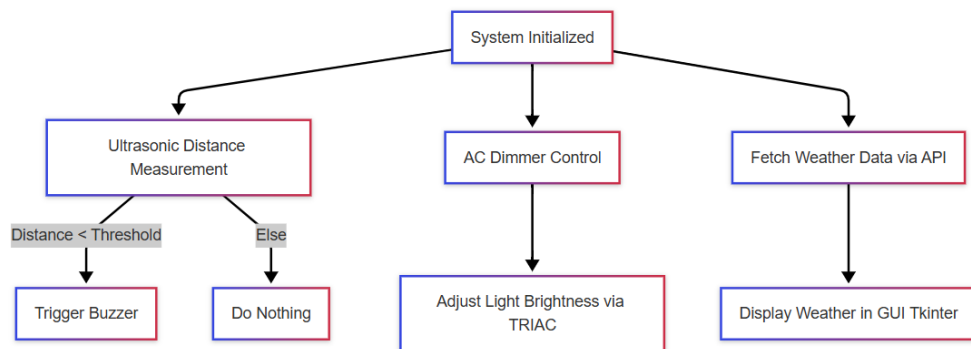


Fig No.49: Additional Sensor-Based Functionalities

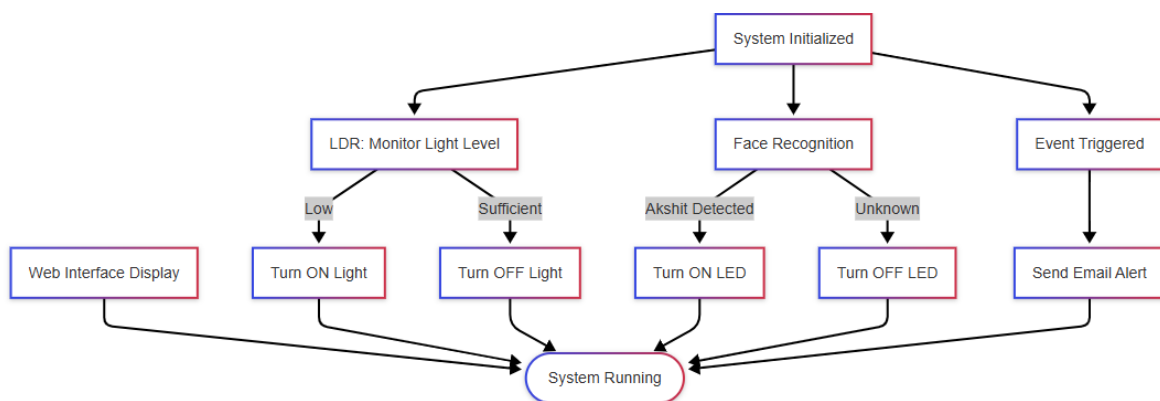


Fig No.50: Smart Automation & Notification Features

4.4.5 Sequence Diagram

The following sequence diagram explains the step-by-step flow when the user accesses the web interface to control the LED.

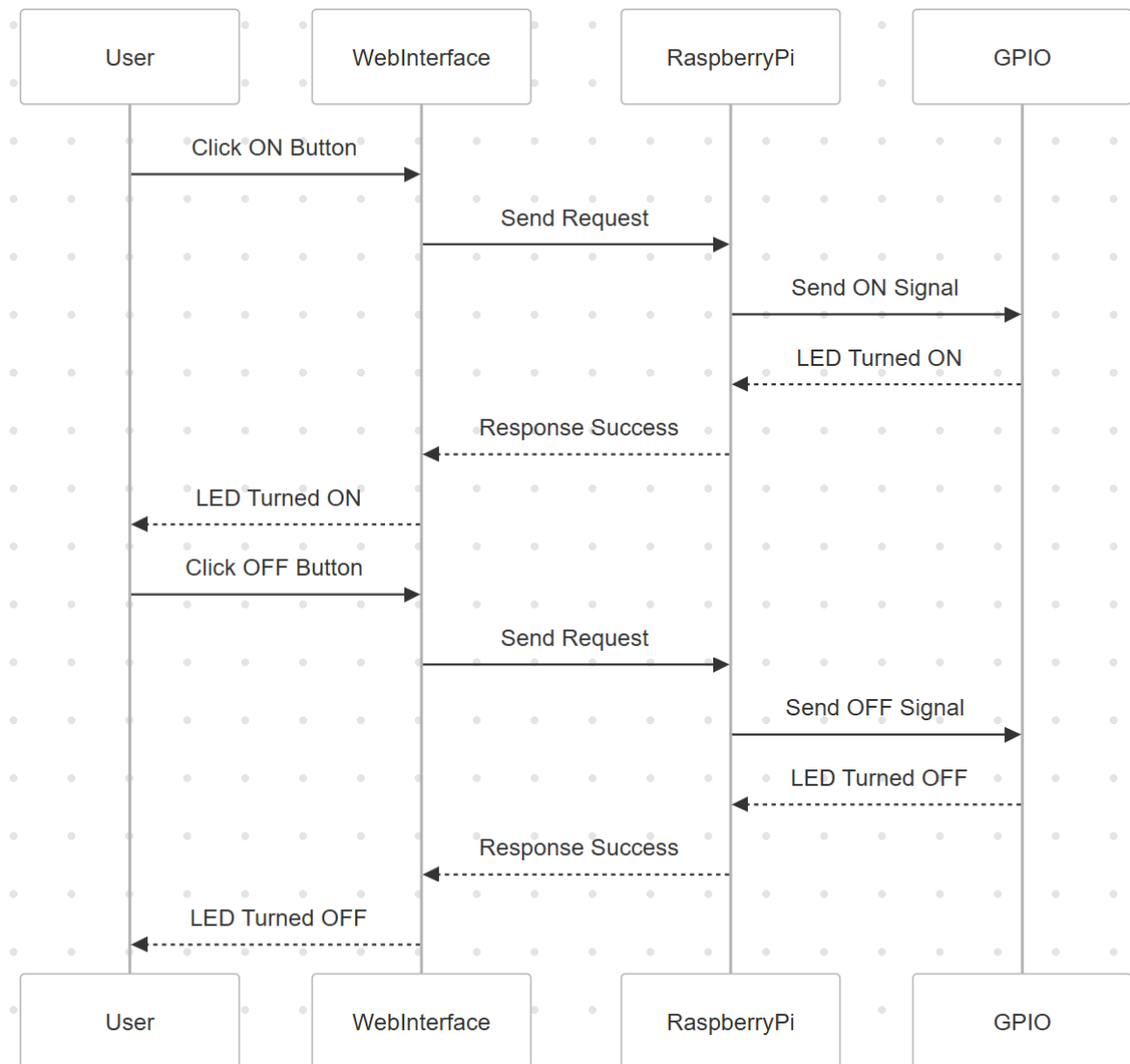


Figure No. 51: Sequence Diagram

Summary

This chapter has undergone a significant expansion to provide a comprehensive overview of the enhanced smart home automation system built using the Raspberry Pi. The discussion now includes detailed explanations of several new features that have been integrated into the system. We have explored the various hardware components that are part of this expanded system, delved into their fundamental working principles, and outlined the overall flow of the code that governs their operation. The addition of components such as the ultrasonic sensor, buzzer, TM1637 display, AC dimmer control, and LDR, coupled with the integration of weather monitoring capabilities and an email alert system, collectively contribute to a substantial enhancement of the system's ability to monitor its environment, control connected devices, and interact with the user. The detailed implementation of the code for these new features is thoroughly documented in the Appendix, offering a complete understanding of the expanded scope and functionality of the project.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

The IoT-based smart home automation system developed in this project demonstrates a comprehensive integration of sensors, actuators, and software frameworks to create a more intelligent and responsive living environment. By utilizing the Raspberry Pi as the central controller, combined with modules such as the PIR motion sensor, DHT11, LDR, ultrasonic sensor, face recognition using PiCamera2, and a Flask-based web interface, the system offers both automation and remote monitoring capabilities.

Key features such as real-time LED control, environmental data display, face recognition-based access, weather updates via OpenWeatherMap API, and email notifications validate the practical applicability of this system for enhancing daily life. This implementation highlights the growing role of edge computing and IoT in building user-centric smart environments that are interactive, secure, and energy-efficient.

The research undertaken throughout the development process shed light on important challenges including sensor integration, real-time data handling, and secure communications. Drawing from prior works ^{[1][2][4][5]}, the project was able to deliver a functional and cost-effective solution that serves as a scalable foundation for more advanced smart home deployments.

However, certain areas present opportunities for future work:

- **System Scalability:** Expanding the platform to support additional smart appliances such as fans, smart locks, or camera-based surveillance.
- **Security Enhancements:** Implementing advanced authentication and encryption mechanisms for web access and device communication.
- **Data Analytics:** Incorporating machine learning algorithms for behavior prediction, automated routines, and optimized energy consumption.

5.1 Future Scope

The project sets the stage for a smarter, safer, and more connected home. Moving forward, the system can be enhanced in the following ways:

- **Smarter Devices:** Add more appliances like smart locks, fans, and surveillance cameras to expand home control.
- **Stronger Security:** Use face + fingerprint access, encrypted data flow, and secure logins to protect the system.
- **Smarter Decisions:** Use AI to learn user habits, predict needs, and reduce energy use automatically.
- **Better Access:** Build a mobile app, enable voice control, and add dashboards for a more user-friendly experience.

- Go Cloud + Edge: Combine cloud storage with real-time local control for speed, safety, and remote access.

In summary, this project lays the groundwork for a robust and versatile smart home system. It showcases the potential of IoT technologies in transforming traditional home environments into adaptive and efficient smart ecosystems, while also emphasizing the need for continuous innovation and refinement in this rapidly evolving field.

References

1. D. Miorandi, S. Sicari, F. De Pellegrini, and I. Chlamtac, "Internet of Things: Vision, applications, and research challenges," *Ad Hoc Networks*, Vol. 10, No. 7, pp. 1497–1516, 2012.
2. A. ElShafee and K. A. Hamed, "Design and Implementation of a WiFi Based Home Automation System," *World Academy of Science, Engineering and Technology*, Vol. 6, No. 8, pp. 2177–2181, 2012.
3. V. R. P. Reddy, A. Sharma, S. R. Kadam, and S. Chavan, "IoT-based Smart Home: Future aspects," *Proceedings of IEEE ANTS*, pp. 1–6, 2017.
4. S. Gupta, R. K. Mishra, and P. Agarwal, "Smart Home Technologies: An IoT-Based Approach," *IoT Journal*, Vol. 3, No. 1, pp. 22–30, 2018.
5. S. Patil, K. D. Yadav, and V. N. Kulkarni, "A Secure IoT Framework for Smart Homes," *International Journal of Computer Applications (IJCA)*, Vol. 5, No. 7, pp. 45–50, 2019.
6. A. Verma and S. Singh, "Home Automation Using Raspberry Pi and IoT," *International Journal of Emerging Technologies (IJET)*, pp. 55–62, 2021.
7. J. Anderson, "Real-Time Monitoring and Automation in Smart Homes," *Proceedings of IoT World Congress*, pp. 50–56, 2019.
8. Li, Y., Mandalari, A.M. and Straw, I., 2023. Who Let the Smart Toaster Hack the House? An Investigation into the Security Vulnerabilities of Consumer IoT Devices. *arXiv preprint*.
9. Magara, T. and Zhou, Y., 2024. Security and Privacy Concerns in the Adoption of IoT Smart Homes: A User-Centric Analysis. *American Journal of Information Science and Technology*, 8(1), pp.1–14.
10. Bouchabou, D., Nguyen, S.M., Lohr, C., Leduc, B. and Kanellos, I., 2021. A Survey of Human Activity Recognition in Smart Homes Based on IoT Sensors Algorithms: Taxonomies, Challenges, and Opportunities with Deep Learning. *arXiv preprint*.
11. Buil-Gil, D., Kemp, S., Kuenzel, S., Coventry, L., Zakhary, S., Tilley, D. and Nicholson, J., 2022. The Digital Harms of Smart Home Devices: A Systematic Literature Review. *arXiv preprint*.
12. Apthorpe, N., Reisman, D., Sundaresan, S., Narayanan, A. and Feamster, N., 2017. Spying on the Smart Home: Privacy Attacks and Defenses on Encrypted IoT Traffic. *arXiv preprint*.
13. Yadav, R., Yadav, N., Gupta, K., Priyadarshini, R., Chakraborty, S. and Kumar, P., 2022. Home Automation Using Internet of Things: An Extensive Review. In: Rawat, S., Kumar, A., Kumar, P. and Anguera, J., eds. *Proceedings of First International Conference on Computational Electronics for Wireless Communications*. Lecture Notes in Networks and Systems, vol. 329. Springer, Singapore, pp.405–414.
14. Singh, U. and Ansari, M.A., 2019. Smart Home Automation System Using Internet of Things. In: *Proceedings of the 2019 2nd International Conference on Power Energy, Environment and Intelligent Control (PEEIC)*. Noida, India, pp.144–149.
15. Davidson, C., Rezwana, T. and Hoque, M.A., 2018. Smart Home Security Application Enabled by IoT. In: *International Conference on Smart Grid and Internet of Things (SGIoT 2018)*. Springer, Cham, pp.46–56.
16. Thomas, D., 2022. IoT-Enabled Smart Home Automation Systems. *Information Sciences and Technological Innovations*, 4(2), p.32.

17. Jabbar, W.A., Kian, T.K., Ramli, R.M., Zubir, S.N., Zamrizaman, N.S.M., Balfaqih, M., Shepelev, V. and Alharbi, S., 2019. Design and Fabrication of Smart Home with Internet of Things Enabled Automation System. *IEEE Access*, 7, pp.144059–144074.
18. Deshpande, C., 2024. *Top 20 Ultimate Internet of Things (IoT) projects for 2024*. [online] Available at: <https://www.simplilearn.com/internet-of-things-iot-projects-article> [Accessed 15 Jan. 2025].
19. Raspberry Pi Foundation, n.d. *Teach, learn, and make with the Raspberry Pi Foundation*. [online] Available at: <https://www.raspberrypi.org/> [Accessed 29 Nov. 2024].
20. MakerBazar, n.d. *Single Channel Relay Module 5V*. [online] Available at: <https://makerbazar.in/products/single-channel-relay-module> [Accessed 12 Dec. 2024].
21. RGRJ AIoT TECHNOLOGIES LLP, 2024. *30Pin CH9102X ESP-32 Development Board with Wifi+Bluetooth - Creating innovative IoT projects*. [online] Available at: <https://aiiottech.in/product/30pin-ch9102x-esp-32-development-board/> [Accessed 7 Jan. 2025].
22. Mermaid Live Editor, n.d. *Online FlowChart & Diagrams Editor - Mermaid Live Editor*. [online] Available at: <https://mermaid.live/edit> [Accessed 10 Dec. 2024].
23. Cd-Team, 2023. *Best 30+ Innovative IoT Projects Ideas for Engineers*. [online] Available at: <https://www.electronicsforu.com/iot-projects-ideas> [Accessed 5 Jan. 2025].
24. Circuit Digest, n.d. *Smart Home Energy Management with IoT and Renewables*. [online] Available at: <https://circuitdigest.com/microcontroller-projects/smart-home-energy-management-with-iot-renewables> [Accessed 18 Nov. 2024].
25. Circuit Digest, n.d. *Voice-Controlled Smart Home Assistant*. [online] Available at: <https://circuitdigest.com/microcontroller-projects/voice-controlled-smart-home-assistant> [Accessed 30 Dec. 2024].
26. GitHub, n.d. *mqtt-smarthome/Architecture.md at master · mqtt-smarthome/mqtt-smarthome*. [online] Available at: <https://github.com/mqtt-smarthome/mqtt-smarthome/blob/master/Architecture.md> [Accessed 1 Jan. 2025].
27. Nosix. (n.d.). *Raspberry GPIO Emulator* [Computer software]. Retrieved from GitHub repository: <https://github.com/nosix/raspberry-gpio-emulator>
28. ElectronicWings, 2025. *HC-SR04 Ultrasonic Sensor Guide with Arduino Interfacing*. [online] Available at: <https://www.electronicwings.com/sensors-modules/ultrasonic-module-hc-sr04> [Accessed 19 Mar. 2025].
29. How To Mechatronics, 2025. *Ultrasonic Sensor HC-SR04 and Arduino - Complete Guide*. [online] Available at: <https://howtomechatronics.com/tutorials/arduino/ultrasonic-sensor-hc-sr04/> [Accessed 6 Jan. 2025].
30. ElectronicWings, 2025. *Ultrasonic Module HC-SR04*. [online] Available at: <https://www.electronicwings.com/sensors-modules/ultrasonic-module-hc-sr04#:~:text=The%20ultrasonic%20sensor%20works%20on,as%20shown%20in%20below%20figure> [Accessed 3 Feb. 2025].
31. Random Nerd Tutorials, 2025. *Complete Guide for Ultrasonic Sensor HC-SR04 with Arduino*. [online] Available at: <https://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/> [Accessed 12 Mar. 2025].
32. Seeed Studio, 2025. *Arduino Ultrasonic Sensor Overview and Tutorials - Latest News*. [online] Available at: <https://www.seeedstudio.com/blog/2019/11/04/hc-sr04-features-arduino-raspberrypi-guide/> [Accessed 17 Feb. 2025].
33. Sparkfun, 2025. *Ultrasonic Ranging Module HC - SR04*. [pdf] Available at: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf> [Accessed 28 Jan. 2025].

34. Vayuya, 2025. *Learn Everything About Ultrasonic Sensor HC SR04*. [online] Available at: <https://vayuya.com/blog/learn-everything-about-ultrasonic-sensor-hc-sr04/> [Accessed 9 Mar. 2025].
35. ElProCus, 2025. *HC-SR04 Ultrasonic Sensor: Pin Configuration, Working and Applications*. [online] Available at: <https://www.elprocus.com/hc-sr04-ultrasonic-sensor-working-and-its-applications/> [Accessed 23 Feb. 2025].
36. SunFounder, 2025. *Ultrasonic Sensor Module (HC-SR04) - Documentation*. [online] Available at: https://docs.sunfounder.com/projects/ultimate-sensor-kit/en/latest/components_basic/01-component_ultrasonic_module.html [Accessed 29 Mar. 2025].
37. ElectroSchematics.com, 2025. *HC-SR04: Datasheet, Specs, and More*. [online] Available at: <https://www.electroschematics.com/hc-sr04-datasheet/> [Accessed 21 Jan. 2025].
38. Handson Technology, 2025. *HC-SR04 Ultrasonic Sensor Module*. [pdf] Available at: <https://handsontec.com/dataspecs/sensor/SR-04-Ultrasonic.pdf> [Accessed 13 Feb. 2025].
39. SparkFun, 2025. *Ultrasonic Distance Sensor - 3.3V (HC-SR04)*. [online] Available at: <https://www.sparkfun.com/ultrasonic-distance-sensor-3-3v-hc-sr04.html> [Accessed 7 Mar. 2025].
40. University of Maine, 2025. *User's Manual – HCSR04 Ultrasonic Sensor*. [pdf] Available at: <https://web.eece.maine.edu/~zhu/book/lab/HC-SR04%20User%20Manual.pdf> [Accessed 2 Apr. 2025].
41. Google Docs, 2025. *HC-SR04 User's Manual*. [online] Available at: https://docs.google.com/document/d/1Y-yZnNhMYy7rwhAgyL_pfa39RsB-x2qR4vP8saG73rE/edit [Accessed 18 Mar. 2025].
42. Mouser, 2025. *HC-SR04 OSEPP Electronics*. [online] Available at: <https://www.mouser.com/ProductDetail/OSEPP-Electronics/HC-SR04?qs=wNBL%252BABd93PqZEhuhHkuOw%3D%3D> [Accessed 5 Jan. 2025].
43. DigiKey, 2025. *HC-SR04 Datasheet by SparkFun Electronics*. [online] Available at: <https://www.digikey.com/htmldatasheets/production/1979760/0/0/1/hc-sr04.html> [Accessed 10 Mar. 2025].
44. Grobotronics, 2025. *HC-SR04 Ultrasonic Sensor - User's Manual*. [pdf] Available at: http://grobotronics.com/images/companies/1/HC-SR04Users_Manual.pdf [Accessed 25 Feb. 2025].
45. Handson Technology, 2025. *HC-SR04 Ultrasonic Sensor Module User Guide*. [pdf] Available at: <https://www.handsontec.com/dataspecs/HC-SR04-Ultrasonic.pdf> [Accessed 30 Jan. 2025].
46. Adafruit, 2025. *HC-SR04 Ultrasonic Sonar Distance Sensor + 2 x 10K resistors*. [online] Available at: <https://www.adafruit.com/product/3942> [Accessed 8 Mar. 2025].
47. PTSolns, 2025. *HC-SR04 Ultrasonic Sensor Updated 2021 Version (4 Pieces)*. [online] Available at: <https://ptsolns.com/products/ultrasonic-sensor> [Accessed 4 Apr. 2025].
48. Adafruit, 2025. *Ultrasonic Distance Sensor - 3V or 5V - HC-SR04 compatible [RCWL-1601]*. [online] Available at: <https://www.adafruit.com/product/4007> [Accessed 17 Jan. 2025].
49. PiBorg, 2025. *Ultrasonic Distance Sensor (HC-SR04)*. [online] Available at: <https://www.piborg.org/sensors-1136/hc-sr04> [Accessed 20 Feb. 2025].
50. Sparkfun, 2025. *Ultrasonic Distance Sensor - 5V (HC-SR04)*. [online] Available at: <https://www.sparkfun.com/ultrasonic-distance-sensor-hc-sr04.html> [Accessed 27 Mar. 2025].
51. Eleobo, 2025. *HC-SR04 Ultrasonic Sensor Guide: Master Applications, Programming, and Arduino Integration for Beginners*. [online] Available at: <https://eleobo.com/how-hc-sr04-ultrasonic-sensor-work-types-of-ultrasonic/> [Accessed 11 Feb. 2025].

52. APMonitor, 2025. *Ultrasonic Distance Measurement HC-SR04 | Data-Driven Engineering*. [online] Available at: <https://apmonitor.com/dde/index.php/Main/UltrasonicDistanceSensor> [Accessed 15 Mar. 2025].
53. Zafeiriou, S., 2025. *HC-SR04 Ultrasonic Sensor Setup: A Complete Guide to Applications, Features, and Setup (2025)*. [online] Available at: <https://stevezafeiriou.com/hc-sr04-ultrasonic-sensor-setup/> [Accessed 1 Apr. 2025].
54. ESPBoards, 2025. *ESP32 HC-SR04 Ultrasonic Distance Sensor Code Examples*. [online] Available at: <https://www.espboards.dev/sensors/hc-sr04/> [Accessed 14 Feb. 2025].
55. Random Nerd Tutorials, 2025. *ESP32 with HC-SR04 Ultrasonic Sensor with Arduino IDE*. [online] Available at: <https://randomnerdtutorials.com/esp32-hc-sr04-ultrasonic-arduino/> [Accessed 6 Mar. 2025].
56. APC International, 2025. *Piezo Buzzers vs. Magnetic Buzzers*. [online] Available at: <https://www.americanpiezo.com/blog/piezo-buzzers-vs-magnetic-buzzers/> [Accessed 22 Jan. 2025].
57. Core Electronics, 2025. *Face Recognition with Raspberry Pi and OpenCV*. [online] Available at: <https://core-electronics.com.au/guides/raspberry-pi/face-recognition-with-raspberry-pi-and-opencv/#code-download> [Accessed 8 Apr. 2025].
58. • Electronics Tutorials. (2025) *Triac – power electronics*. Available at: <https://www.electronics-tutorials.ws/power/triac.html> (Accessed: 01 January 2025).
59. • Mastrofini, A. (2023) *Read a TRIAC controller for IoT*. Available at: <https://alessandromastrofini.it/2023/05/13/read-a-triac-controller-for-iot/> (Accessed: 05 January 2025).
60. • Research Design Lab. (2025) *Dimmer module*. Available at: <https://researchdesignlab.com/dimmer-module.html> (Accessed: 09 January 2025).
61. • University of Washington DXARTS. (2025) *AC dimmer circuit*. Available at: <https://dxarts.washington.edu/wiki/ac-dimmer-circuit> (Accessed: 13 January 2025).
62. • Rajguru Electronics. (2025) *LDR Module Datasheet*. Available at: https://www.rajguruelectronics.com/Product/8749/A87493_LDR%20Module_Datasheet.pdf (Accessed: 17 January 2025).
63. • Elprocus. (2025) *5V Relay Module*. Available at: <https://www.elprocus.com/5v-relay-module/> (Accessed: 21 January 2025).
64. • Last Minute Engineers. (2025) *PIR Sensor Arduino Tutorial*. Available at: <https://lastminuteengineers.com/pir-sensor-arduino-tutorial/> (Accessed: 25 January 2025).
65. Seeed Studio, 2025. *Introduction to Buzzers: Piezo and Magnetic buzzers*. [online] Available at: <https://www.seeedstudio.com/blog/2020/12/22/introduction-to-buzzers-piezo-and-magnetic-buzzers/> [Accessed 6 Jan. 2025].
66. STEMpedia Education, 2025. *Digital Clock Using TM1637 Display and RTC PCF8563*. [online] Available at: <https://ai.thestempedia.com/docs/evive/evive-arduino-ide-tutorials/making-digital-clock-using-tm1637-4-digit-display/> [Accessed 22 Jan. 2025].
67. SuperLightingLED, 2025. *A Quick And Deep Understanding Of TRIAC Dimmer & Dimming – SuperLightingLED Blog*. [online] Available at: <https://www.superlightingled.com/blog/a-quick-and-deep-understanding-of-triac-dimmer-dimming/> [Accessed 3 Mar. 2025].
68. ThinkRobotics.com, 2025. *LDR Sensor: A Comprehensive Guide to Light-Dependent Resistors*. [online] Available at: <https://thinkrobotics.com/blogs/learn/ldr-sensor-a-comprehensive-guide-to-light-dependent-resistors> [Accessed 9 Feb. 2025].

Appendix A:

1. Sensor Control Code

```
import tkinter as tk
from tkinter import ttk
import threading
import time
from sensor_monitor import SensorMonitor

class SensorControl:
    def __init__(self, root):
        self.root = root
        self.root.title("Sensor Control Panel")
        self.root.geometry("800x600")

        # Create main frame
        self.main_frame = ttk.Frame(root, padding="10")
        self.main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        # Status variables
        self.monitoring = False
        self.sensor_monitor = None
        self.monitor_thread = None

        # Create control buttons
        self.create_control_buttons()

        # Create settings frame
        self.create_settings_frame()

        # Create status display
        self.create_status_display()

    def create_control_buttons(self):
        # Control buttons frame
        control_frame = ttk.LabelFrame(self.main_frame, text="Control",
padding="5")
        control_frame.grid(row=0, column=0, padx=5, pady=5, sticky=(tk.W,
tk.E))

        # Start button
        self.start_button = ttk.Button(control_frame, text="Start Monitoring",
command=self.start_monitoring)
        self.start_button.grid(row=0, column=0, padx=5, pady=5)

        # Stop button
        self.stop_button = ttk.Button(control_frame, text="Stop Monitoring",
command=self.stop_monitoring, state=tk.DISABLED)
```

```

        self.stop_button.grid(row=0, column=1, padx=5, pady=5)

    def create_settings_frame(self):
        # Settings frame
        settings_frame = ttk.LabelFrame(self.main_frame, text="Settings",
padding="5")
        settings_frame.grid(row=1, column=0, padx=5, pady=5, sticky=(tk.W,
tk.E))

        # Motion detection sensitivity
        ttk.Label(settings_frame, text="Motion Sensitivity:").grid(row=0,
column=0, padx=5, pady=5)
        self.sensitivity_var = tk.StringVar(value="Medium")
        sensitivity_combo = ttk.Combobox(settings_frame,
textvariable=self.sensitivity_var,
                                values=["Low", "Medium", "High"],
state="readonly")
        sensitivity_combo.grid(row=0, column=1, padx=5, pady=5)

        # Update interval
        ttk.Label(settings_frame, text="Update Interval
(seconds):").grid(row=1, column=0, padx=5, pady=5)
        self.interval_var = tk.StringVar(value="1")
        interval_entry = ttk.Entry(settings_frame,
textvariable=self.interval_var, width=10)
        interval_entry.grid(row=1, column=1, padx=5, pady=5)

        # Save settings button
        ttk.Button(settings_frame, text="Save Settings",
command=self.save_settings).grid(row=2, column=0, columnspan=2, pady=5)

    def create_status_display(self):
        # Status display frame
        status_frame = ttk.LabelFrame(self.main_frame, text="Status",
padding="5")
        status_frame.grid(row=2, column=0, padx=5, pady=5, sticky=(tk.W, tk.E,
tk.N, tk.S))

        # Status text
        self.status_text = tk.Text(status_frame, height=10, width=70)
        self.status_text.grid(row=0, column=0, padx=5, pady=5)
        self.status_text.insert(tk.END, "System ready. Click 'Start
Monitoring' to begin.\n")
        self.status_text.config(state=tk.DISABLED)

    def start_monitoring(self):
        if not self.monitoring:
            self.monitoring = True

```

```

        self.start_button.config(state=tk.DISABLED)
        self.stop_button.config(state=tk.NORMAL)
        # Create and start sensor monitor in a separate thread
        monitor_root = tk.Tk()
        self.sensor_monitor = SensorMonitor(monitor_root)
        self.monitor_thread =
threading.Thread(target=monitor_root.mainloop, daemon=True)
        self.monitor_thread.start()

        self.update_status("Monitoring started")

def stop_monitoring(self):
    if self.monitoring:
        self.monitoring = False
        self.start_button.config(state=tk.NORMAL)
        self.stop_button.config(state=tk.DISABLED)

        # Stop the sensor monitor
        if self.sensor_monitor:
            self.sensor_monitor.cleanup()
            self.sensor_monitor.root.quit()
            self.sensor_monitor = None

        self.update_status("Monitoring stopped")

def save_settings(self):
    try:
        interval = float(self.interval_var.get())
        if interval <= 0:
            raise ValueError("Interval must be positive")

        sensitivity = self.sensitivity_var.get()
        self.update_status(f"Settings saved:\n- Update Interval:
{interval} seconds\n- Sensitivity: {sensitivity}")

    except ValueError as e:
        self.update_status(f"Error: {str(e)}")

def update_status(self, message):
    self.status_text.config(state=tk.NORMAL)
    self.status_text.insert(tk.END, f"{message}\n")
    self.status_text.see(tk.END)
    self.status_text.config(state=tk.DISABLED)

if __name__ == "__main__":
    root = tk.Tk()
    app = SensorControl(root)
    root.mainloop()

```

2. Weather API Code

```
import tkinter as tk
from tkinter import messagebox, PhotoImage
import requests
from PIL import Image, ImageTk
from io import BytesIO
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

API_KEY = "847f127b6144xxxxxxxxx" # Replace with your OpenWeatherMap API key
CURRENT_URL = "https://api.openweathermap.org/data/2.5/weather"
FORECAST_URL = "https://api.openweathermap.org/data/2.5/forecast"

# Theme colors
light_theme = {
    "bg": "#f0f0f0",
    "fg": "#000000",
    "entry": "#ffffff",
    "button": "#d9d9d9"
}

dark_theme = {
    "bg": "#2c2f33",
    "fg": "#ffffff",
    "entry": "#40444b",
    "button": "#7289da"
}

theme = light_theme # default

def detect_city():
    try:
        res = requests.get("http://ip-api.com/json/").json()
        return res.get("city", "Delhi")
    except:
        return "Delhi"

def kelvin_to_celsius(k):
    return round(k - 273.15, 1)

def get_weather(city):
    params = {'q': city, 'appid': API_KEY, 'units': 'metric'}
    current = requests.get(CURRENT_URL, params=params).json()
    forecast = requests.get(FORECAST_URL, params=params).json()
    return current, forecast

def display_weather():
```

```

city = city_entry.get().strip()
if not city:
    messagebox.showwarning("Input Error", "Please enter a city name.")
    return

try:
    current, forecast = get_weather(city)

    if current.get("cod") != 200:
        result_label.config(text="✗ City not found.")
        forecast_label.config(text="")
        return

    # Current weather
    weather_desc = current['weather'][0]['description'].title()
    temp = current['main']['temp']
    humidity = current['main']['humidity']
    wind = current['wind']['speed']

    # Icon
    icon_code = current['weather'][0]['icon']
    icon_url = f"http://openweathermap.org/img/wn/{icon_code}@2x.png"
    icon_data = requests.get(icon_url).content
    img = Image.open(BytesIO(icon_data))
    img = img.resize((80, 80))
    photo = ImageTk.PhotoImage(img)
    icon_label.config(image=photo)
    icon_label.image = photo

    current_text = f"""
📍 {city.title()}
☁️ {weather_desc}
🌡️ Temp: {temp} °C
💧 Humidity: {humidity}%
💨 Wind: {wind} m/s
    """
    result_label.config(text=current_text.strip())

    # Forecast
    forecast_list = forecast["list"]
    daily_forecast = []
    temps = []
    humidity_vals = []
    labels = []

    for i in range(0, 40, 8):
        day = forecast_list[i]
        day_temp = day['main']['temp']

```

```

        hum = day['main']['humidity']
        day_desc = day['weather'][0]['description'].title()
        date = day['dt_txt'].split()[0]

        daily_forecast.append(f"{date}: 🌡️ {day_temp} °C | {day_desc}")
        temps.append(day_temp)
        humidity_vals.append(hum)
        labels.append(date)

    forecast_label.config(text="\n".join(daily_forecast))

    # Plot Graph
    plot_graph(labels, temps, humidity_vals)

except Exception as e:
    result_label.config(text="❌ Error fetching weather.")
    forecast_label.config(text="")
    print(e)

def toggle_theme():
    global theme
    theme = dark_theme if theme == light_theme else light_theme
    apply_theme()

def apply_theme():
    root.configure(bg=theme["bg"])
    for widget in root.winfo_children():
        if isinstance(widget, (tk.Label, tk.Button)):
            widget.configure(bg=theme["bg"], fg=theme["fg"])
        if isinstance(widget, tk.Entry):
            widget.configure(bg=theme["entry"], fg=theme["fg"])
    city_entry.configure(insertbackground=theme["fg"])

def plot_graph(labels, temps, humidity_vals):
    for widget in graph_frame.winfo_children():
        widget.destroy()

    fig, ax1 = plt.subplots(figsize=(4.5, 2.5), dpi=100)
    ax1.plot(labels, temps, label="Temperature (°C)", marker='o',
color='orange')
    ax1.set_ylabel("Temp (°C)")
    ax1.set_xlabel("Date")
    ax1.grid(True)
    ax1.tick_params(axis='x', rotation=45)
    ax1.legend(loc="upper left")

    ax2 = ax1.twinx()

```

```

    ax2.plot(labels, humidity_vals, label="Humidity (%)", marker='s',
color='blue')
    ax2.set_ylabel("Humidity (%)")
    ax2.legend(loc="upper right")

    fig.tight_layout()
    canvas = FigureCanvasTkAgg(fig, master=graph_frame)
    canvas.draw()
    canvas.get_tk_widget().pack()

# GUI setup
root = tk.Tk()
root.title("☁️ Weather Dashboard")
root.geometry("500x700")
root.resizable(False, False)

detected_city = detect_city()
tk.Label(root, text="Enter City", font=("Helvetica", 14)).pack(pady=10)
city_entry = tk.Entry(root, font=("Helvetica", 16), width=30)
city_entry.insert(0, detected_city)
city_entry.pack(pady=5)

tk.Button(root, text="🔍 Get Weather", font=("Helvetica", 12),
command=display_weather).pack(pady=10)
tk.Button(root, text="🔄 Toggle Theme", font=("Helvetica", 12),
command=toggle_theme).pack()

icon_label = tk.Label(root, bg=theme["bg"])
icon_label.pack()

result_label = tk.Label(root, text="", font=("Helvetica", 12), justify="left",
wraplength=450)
result_label.pack(pady=10)

tk.Label(root, text="📊 5-Day Forecast", font=("Helvetica", 13,
"bold")).pack(pady=5)
forecast_label = tk.Label(root, text="", font=("Helvetica", 11),
justify="left", wraplength=450)
forecast_label.pack(pady=10)

graph_frame = tk.Frame(root, bg=theme["bg"])
graph_frame.pack(pady=10)

apply_theme()
root.mainloop()

```

3. Ultrasonic Sensor Code

```
# Import required libraries
import RPi.GPIO as GPIO
import time

# GPIO Pin Configuration
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

# Define GPIO pins
GPIO_TRIGGER = 18
GPIO_ECHO = 24
BUZZER = 23 # Connect the buzzer to GPIO 23

# Setup GPIO direction (IN/OUT)
GPIO.setup(GPIO_TRIGGER, GPIO.OUT)
GPIO.setup(GPIO_ECHO, GPIO.IN)
GPIO.setup(BUZZER, GPIO.OUT)

def get_distance(samples=3):
    """
    Measures distance using the ultrasonic sensor, averaging multiple samples
    for stability.
    """
    distances = []

    for _ in range(samples):
        # Send trigger pulse
        GPIO.output(GPIO_TRIGGER, True)
        time.sleep(0.00001)
        GPIO.output(GPIO_TRIGGER, False)

        start_time = time.time()
        stop_time = time.time()

        # Record the time of the echo signal start
        while GPIO.input(GPIO_ECHO) == 0:
            start_time = time.time()

        # Record the time of the echo signal end
        while GPIO.input(GPIO_ECHO) == 1:
            stop_time = time.time()

        # Compute distance in cm
        time_elapsed = stop_time - start_time
        distance = (time_elapsed * 34300) / 2
        distances.append(distance)
```

```

        time.sleep(0.05) # Short delay between samples

# Return the averaged distance
return sum(distances) / len(distances)

def buzzer_control(distance):
    """
    Activates the buzzer if the measured distance exceeds 3 cm.
    """
    if distance > 3:
        GPIO.output(BUZZER, False) # Turn buzzer ON
    else:
        GPIO.output(BUZZER, True) # Turn buzzer OFF

if __name__ == '__main__':
    try:
        print("\nUltrasonic Sensor Initialized")
        print("Testing all components...\n")
        time.sleep(2)

        while True:
            dist = get_distance()
            print(f"Measured Distance: {dist:.2f} cm")

            # Control buzzer based on distance
            buzzer_control(dist)

            time.sleep(1) # Adjust delay as needed

    except KeyboardInterrupt:
        print("\nMeasurement stopped by User")

    except Exception as e:
        print(f"\nAn error occurred: {e}")

    finally:
        GPIO.output(BUZZER, False) # Ensure buzzer is turned off
        GPIO.cleanup()
        print("GPIO cleaned up. Exiting...")

```

4. Temperature Display Code

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

from time import sleep
import tm1637
import Adafruit_DHT

sensor = Adafruit_DHT.DHT11
pin = 4 # Pin defined for data input
# DHT11 pin4 = Data; VCC = 3V3; GND = Ground
# TM1637 VCC = 5V; GND = Ground; DIO = 20; CLK = 21

# Initialize the display (GND, VCC=3.3V, Example Pins are DIO-20 and CLK21)
Display = tm1637.TM1637(CLK=21, DIO=20, brightness=1.0)

def measure_temperature():
    """Measure temperature and humidity, return values rounded to 2 decimal
    places."""
    humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)
    if humidity is not None and temperature is not None:
        return round(temperature, 2), round(humidity, 2)
    else:
        return None, None

def display_temperature(temp):
    """Display integer part of temperature on 4-digit 7-segment display,
    separated by colon."""
    if temp is not None:
        temp_int = int(temp) # Extract integer part
        digits = [int(d) for d in f"{temp_int:02d}00"] # Format to display 2-
        digit number with colon
        Display.Show(digits)
        Display.ShowDoublepoint(True)
    else:
        Display.Show([0, 0, 0, 0]) # Show blank in case of invalid reading

def main():
    print("Program for Temperature Sensor (DHT11)")
    try:
        # Display digits from 1 to 9 sequentially
        for i in range(1, 10):
            Display.Show([0, 0, 0, i]) # Display single digit at the right-
            most position
            sleep(0.5)

        while True:
```

```

temp, humidity = measure_temperature()
if temp is not None and humidity is not None:
    print(f'Temp={temp:.2f}*C Humidity={humidity:.2f}%')
    x = (int(temp) * 100) + int(humidity)
    # print(x) Temperature and humidity concatenate
    display_temperature(x)
else:
    print("Failed to get valid readings. Retrying...")
    sleep(2)
except KeyboardInterrupt:
    print("\nProgram interrupted by user.")
except Exception as e:
    print(f"An error occurred: {e}")
finally:
    print("Cleaning up and exiting program.")
    Display.cleanup()

if __name__ == "__main__":
    main()

```

5. SMTP Function Code

```

send_email(
    sender_email="raspberrypixxxx@gmail.com",
    app_password="gjkidgjcxxxxxxx",
    recipient_email="examplexxx.in",
    subject="Test from Raspberry Pi 5",
    body="Hey, this is a test email sent from my Raspberry Pi!"
)

```

6. AC Dimmer Code

```
import RPi.GPIO as GPIO
import time

# Define GPIO pins
AC_LOAD = 18 # GPIO pin to control the dimmer module
ZERO_CROSS = 17 # GPIO pin for zero-cross detection

# Initial brightness level (0-100%)
dimming = 100

def zero_cross_detect(channel):
    """Interrupt function triggered at each zero-crossing."""
    dim_time = int(75 * (100 - dimming)) # Convert dimming % to delay time
    time.sleep(dim_time / 1_000_000) # Convert microseconds to seconds
    GPIO.output(AC_LOAD, GPIO.HIGH)
    time.sleep(10 / 1_000_000) # 10µs pulse to fire the TRIAC
    GPIO.output(AC_LOAD, GPIO.LOW)

def setup():
    """Setup GPIO and attach zero-cross interrupt."""
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(AC_LOAD, GPIO.OUT)
    GPIO.setup(ZERO_CROSS, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.add_event_detect(ZERO_CROSS, GPIO.RISING, callback=zero_cross_detect)

def adjust_dimming(level):
    """Adjust dimming level smoothly."""
    global dimming
    if 0 <= level <= 100:
        dimming = level
        print(f"Dimming Level Set To: {dimming}%")
    else:
        print("Invalid dimming level! Please enter a value between 0 and 100.")

def user_control():
    """Allow user to control brightness manually."""
    global dimming
    try:
        while True:
            user_input = input("Enter dimming level (0-100) or 'exit' to quit: ")
            user_input = user_input.strip()
            if user_input.lower() == 'exit':
                break
```

```

        if user_input.isdigit():
            adjust_dimming(int(user_input))
        else:
            print("Invalid input! Please enter a number between 0 and
100.")
    except KeyboardInterrupt:
        print("\nUser exited.")

def cleanup():
    """Cleanup GPIO before exiting."""
    GPIO.output(AC_LOAD, GPIO.LOW)
    GPIO.cleanup()
    print("GPIO cleaned up. Exiting...")

if __name__ == '__main__':
    try:
        setup()
        print("\nAC Dimmer Control Initialized.")
        print("Default brightness: 100%\n")
        user_control()
    except Exception as e:
        print(f"An error occurred: {e}")
    finally:
        cleanup()

```

7. LDR Code

```

import RPi.GPIO as GPIO
import time

# Define GPIO pins
# LDR 3v3, GND
LDR_PIN = 26 # Change based on your connection
# Relay 5v, GND
LED_PIN = 19 # GPIO pin for LED/ Relay

# GPIO setup
GPIO.setmode(GPIO.BCM)
GPIO.setup(LDR_PIN, GPIO.IN)
GPIO.setup(LED_PIN, GPIO.OUT)

def read_ldr():
    """
    Reads the LDR sensor and controls the LED accordingly.
    Only prints when there is a state change (avoids redundant output).
    """
    prev_state = None # Stores the previous light status

```

```

while True:
    try:
        light_status = GPIO.input(LDR_PIN)
        if light_status != prev_state: # Print only if state changes
            if light_status == 0:
                print("Bright Light Detected!")
                GPIO.output(LED_PIN, GPIO.HIGH) # Turn LED ON
            else:
                print("Darkness Detected!")
                GPIO.output(LED_PIN, GPIO.LOW) # Turn LED OFF
            prev_state = light_status # Update previous state
        time.sleep(0.5) # Small delay to avoid excessive polling
    except KeyboardInterrupt:
        print("\nProgram Terminated by User.")
        cleanup()
        break
    except Exception as e:
        print(f"⚠ Error Occurred: {e}")
        cleanup()
        break

def cleanup():
    """Clean up GPIO before exiting."""
    GPIO.output(LED_PIN, GPIO.LOW) # Ensure LED is off before exit
    GPIO.cleanup()
    print("GPIO Cleaned Up. Exiting...")

if __name__ == '__main__':
    print("\nLDR Sensor Monitoring Started. Press Ctrl+C to exit.")
    read_ldr()

```

8. SMTP Code

```
import smtplib
from email.message import EmailMessage

# === Email Configuration ===

EMAIL_ADDRESS = "raspberrypixxxx@gmail.com"      # <-- Enter your Gmail
address here
EMAIL_PASSWORD = "gjkidgjalxxxxxx"              # <-- Enter your Gmail App
Password here (not your Gmail password)
TO_EMAIL = "xxxx.in"                             # <-- Enter the recipient's email address here

def send_email_alert(subject, body):
    """Function to send an email alert with a subject and body."""
    msg = EmailMessage()
    msg.set_content(body)
    msg["Subject"] = subject
    msg["From"] = EMAIL_ADDRESS
    msg["To"] = TO_EMAIL

    try:
        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
            server.login(EMAIL_ADDRESS, EMAIL_PASSWORD) # Log in to Gmail
SMTP server
            server.send_message(msg)                   # Send the email
            print("☑ Email sent successfully!")
    except Exception as e:
        print(f"✗ Failed to send email: {e}")

# === Test email send (optional) ===
# You can run this script directly to test email sending.
if __name__ == "__main__":
    send_email_alert(
        "Test Email from Raspberry Pi",              # <-- Enter
your test subject
        "This is a test email sent from my Raspberry Pi!" # <-- Enter
your test message
    )
```

9. Sensor Monitor Code

```
import tkinter as tk
from tkinter import ttk
import sqlite3
from datetime import datetime
import threading
import time
from EmulatorGUI import GPIO

class SensorMonitor:
    def __init__(self, root):
        self.root = root
        self.root.title("Sensor Monitoring System")
        self.root.geometry("800x600")

        # Initialize GPIO
        GPIO.setmode(GPIO.BCM)
        GPIO.setwarnings(False)

        # Setup pins
        self.LED_PIN = 18
        self.PIR_PIN = 17
        GPIO.setup(self.LED_PIN, GPIO.OUT)
        GPIO.setup(self.PIR_PIN, GPIO.IN)

        # Initialize variables
        self.led_status = False
        self.monitoring = False

        # Initialize database
        self.init_database()

        # Create main frame
        self.main_frame = ttk.Frame(root, padding="10")
        self.main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        # Create tabs
        self.tab_control = ttk.Notebook(self.main_frame)

        # Motion Tab
        self.motion_tab = ttk.Frame(self.tab_control)
        self.tab_control.add(self.motion_tab, text='Motion')

        # LED Control Tab
        self.led_tab = ttk.Frame(self.tab_control)
        self.tab_control.add(self.led_tab, text='LED Control')

        # History Tab
```

```

self.history_tab = ttk.Frame(self.tab_control)
self.tab_control.add(self.history_tab, text='History')

self.tab_control.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N,
tk.S))

# Initialize displays
self.init_motion_display()
self.init_led_display()
self.init_history_display()

# Start sensor monitoring thread
self.start_sensor_threads()

def cleanup(self):
    self.monitoring = False
    GPIO.cleanup()

def init_database(self):
    conn = sqlite3.connect('sensor_data.db')
    cursor = conn.cursor()

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS motion_data (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp DATETIME,
            motion_detected INTEGER
        )
    ''')

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS led_data (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp DATETIME,
            led_state INTEGER
        )
    ''')

    conn.commit()
    conn.close()

def init_motion_display(self):
    # Motion display frame
    motion_frame = ttk.Frame(self.motion_tab)
    motion_frame.grid(row=0, column=0, padx=10, pady=10, sticky=(tk.W,
tk.E, tk.N, tk.S))

    # Status section

```

```

        status_frame = ttk.LabelFrame(motion_frame, text="Motion Status",
padding="10")
        status_frame.grid(row=0, column=0, padx=5, pady=5, sticky=(tk.W,
tk.E))

        # Motion status with larger font and color indicator
        self.motion_label = ttk.Label(status_frame, text="No Motion",
font=('Arial', 14, 'bold'))
        self.motion_label.grid(row=0, column=0, padx=5, pady=5)

        # Status indicator (colored circle)
        self.status_canvas = tk.Canvas(status_frame, width=20, height=20)
        self.status_canvas.grid(row=0, column=1, padx=5, pady=5)
        self.status_indicator = self.status_canvas.create_oval(2, 2, 18, 18,
fill='red')

        # Controls section
        control_frame = ttk.LabelFrame(motion_frame, text="Controls",
padding="10")
        control_frame.grid(row=1, column=0, padx=5, pady=5, sticky=(tk.W,
tk.E))

        # Sensitivity control
        ttk.Label(control_frame, text="Sensitivity:").grid(row=0, column=0,
padx=5, pady=5)
        self.sensitivity_var = tk.StringVar(value="Medium")
        sensitivity_combo = ttk.Combobox(control_frame,
textvariable=self.sensitivity_var,
                                values=["Low", "Medium", "High"],
state="readonly", width=15)
        sensitivity_combo.grid(row=0, column=1, padx=5, pady=5)

        # Update interval control
        ttk.Label(control_frame, text="Update Interval (s):").grid(row=1,
column=0, padx=5, pady=5)
        self.interval_var = tk.StringVar(value="1.0")
        interval_entry = ttk.Entry(control_frame,
textvariable=self.interval_var, width=10)
        interval_entry.grid(row=1, column=1, padx=5, pady=5)

        # Statistics section
        stats_frame = ttk.LabelFrame(motion_frame, text="Statistics",
padding="10")
        stats_frame.grid(row=2, column=0, padx=5, pady=5, sticky=(tk.W, tk.E))

        # Detection counter
        ttk.Label(stats_frame, text="Total Detections:").grid(row=0, column=0,
padx=5, pady=5)

```

```

        self.detection_count = tk.StringVar(value="0")
        ttk.Label(stats_frame, textvariable=self.detection_count).grid(row=0,
column=1, padx=5, pady=5)

        # Last detection time
        ttk.Label(stats_frame, text="Last Detection:").grid(row=1, column=0,
padx=5, pady=5)
        self.last_detection_time = tk.StringVar(value="N/A")
        ttk.Label(stats_frame,
textvariable=self.last_detection_time).grid(row=1, column=1, padx=5, pady=5)

        # Reset statistics button
        ttk.Button(stats_frame, text="Reset Statistics",
                    command=self.reset_motion_stats).grid(row=2, column=0,
columnspan=2, pady=10)

        # Initialize statistics variables
        self.total_detections = 0

    def init_led_display(self):
        # LED control
        ttk.Label(self.led_tab, text="LED Status:").grid(row=0, column=0,
padx=5, pady=5)
        self.led_status_label = ttk.Label(self.led_tab, text="OFF")
        self.led_status_label.grid(row=0, column=1, padx=5, pady=5)

        self.toggle_button = ttk.Button(self.led_tab, text="Toggle LED",
command=self.toggle_led)
        self.toggle_button.grid(row=1, column=0, columnspan=2, pady=5)

    def init_history_display(self):
        # History display
        self.history_tree = ttk.Treeview(self.history_tab,
columns=('Timestamp', 'Sensor', 'Value'), show='headings')
        self.history_tree.heading('Timestamp', text='Timestamp')
        self.history_tree.heading('Sensor', text='Sensor')
        self.history_tree.heading('Value', text='Value')
        self.history_tree.grid(row=0, column=0, padx=5, pady=5)

        # Add scrollbar
        scrollbar = ttk.Scrollbar(self.history_tab, orient=tk.VERTICAL,
command=self.history_tree.yview)
        scrollbar.grid(row=0, column=1, sticky='ns')
        self.history_tree.configure(yscrollcommand=scrollbar.set)

    def start_sensor_threads(self):
        # Start motion monitoring thread
        self.monitoring = True

```

```

        motion_thread = threading.Thread(target=self.monitor_motion,
daemon=True)
        motion_thread.start()

def reset_motion_stats(self):
    self.total_detections = 0
    self.detection_count.set("0")
    self.last_detection_time.set("N/A")
    self.update_history('Motion', "Statistics Reset")

def monitor_motion(self):
    time.sleep(10) # Allow sensor to settle

    while self.monitoring:
        motion_detected = GPIO.input(self.PIR_PIN)
        status = "Motion Detected" if motion_detected else "No Motion"

        # Update motion label and indicator
        self.motion_label.config(text=status)
        self.status_canvas.itemconfig(self.status_indicator,
fill='green' if motion_detected else
'red')

        # Update statistics if motion is detected
        if motion_detected:
            self.total_detections += 1
            self.detection_count.set(str(self.total_detections))
            current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
            self.last_detection_time.set(current_time)

        # Store in database
        self.store_motion_data(motion_detected)

        # Update history
        self.update_history('Motion', status)

        # Use the interval from the GUI
        try:
            interval = float(self.interval_var.get())
            if interval <= 0:
                interval = 1.0
        except ValueError:
            interval = 1.0

        time.sleep(interval)

def toggle_led(self):
    self.led_status = not self.led_status

```

```

GPIO.output(self.LED_PIN, GPIO.HIGH if self.led_status else GPIO.LOW)
self.led_status_label.config(text="ON" if self.led_status else "OFF")

# Store in database
self.store_led_data(self.led_status)

# Update history
self.update_history('LED', "ON" if self.led_status else "OFF")

def store_motion_data(self, motion_detected):
    conn = sqlite3.connect('sensor_data.db')
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO motion_data (timestamp, motion_detected)
        VALUES (?, ?)
    ''', (datetime.now(), motion_detected))
    conn.commit()
    conn.close()

def store_led_data(self, led_state):
    conn = sqlite3.connect('sensor_data.db')
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO led_data (timestamp, led_state)
        VALUES (?, ?)
    ''', (datetime.now(), led_state))
    conn.commit()
    conn.close()

def update_history(self, sensor, value):
    self.history_tree.insert('', 'end',
values=(datetime.now().strftime('%Y-%m-%d %H:%M:%S'), sensor, value))
    # Keep only last 100 entries
    if len(self.history_tree.get_children()) > 100:
        self.history_tree.delete(self.history_tree.get_children()[0])

if __name__ == "__main__":
    root = tk.Tk()
    app = SensorMonitor(root)
    root.mainloop()

```